Machine Learning
Deep Generative Models

Rajesh Ranganath

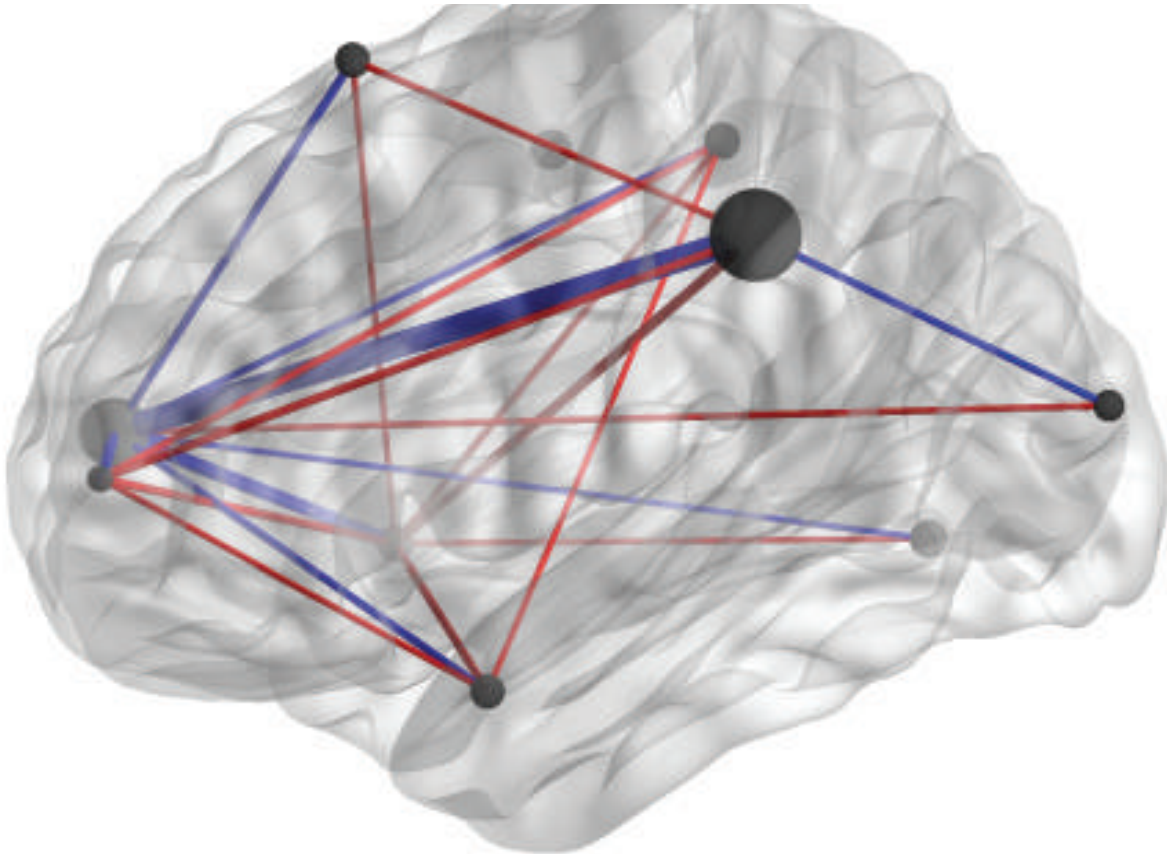
Last Class

We talked about building models

$$\int p(\mathbf{x}, \mathbf{z}) d\mathbf{z} = p(\mathbf{x})$$

Models can be used to find hidden structure

Neuroscience



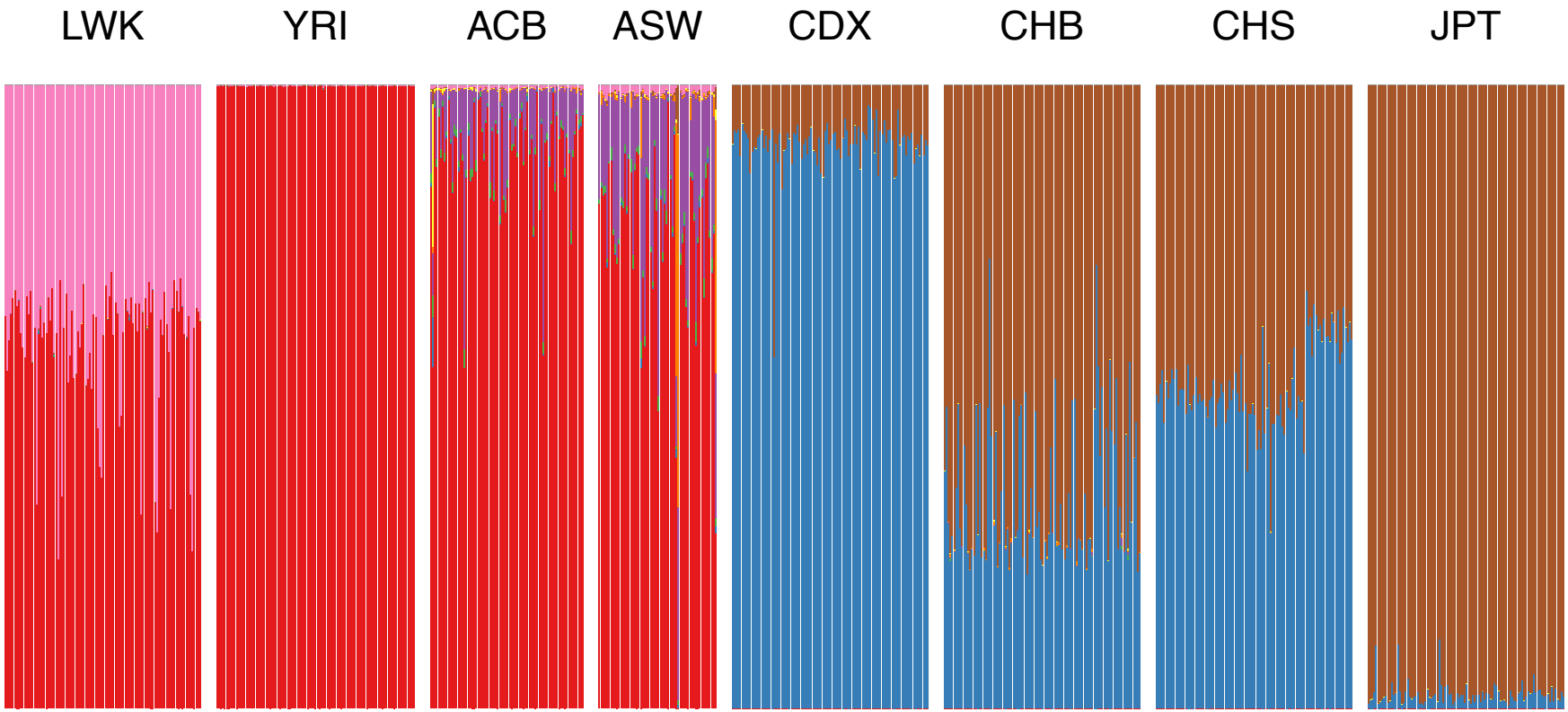
[Manning+ 2014]

Astrophysics



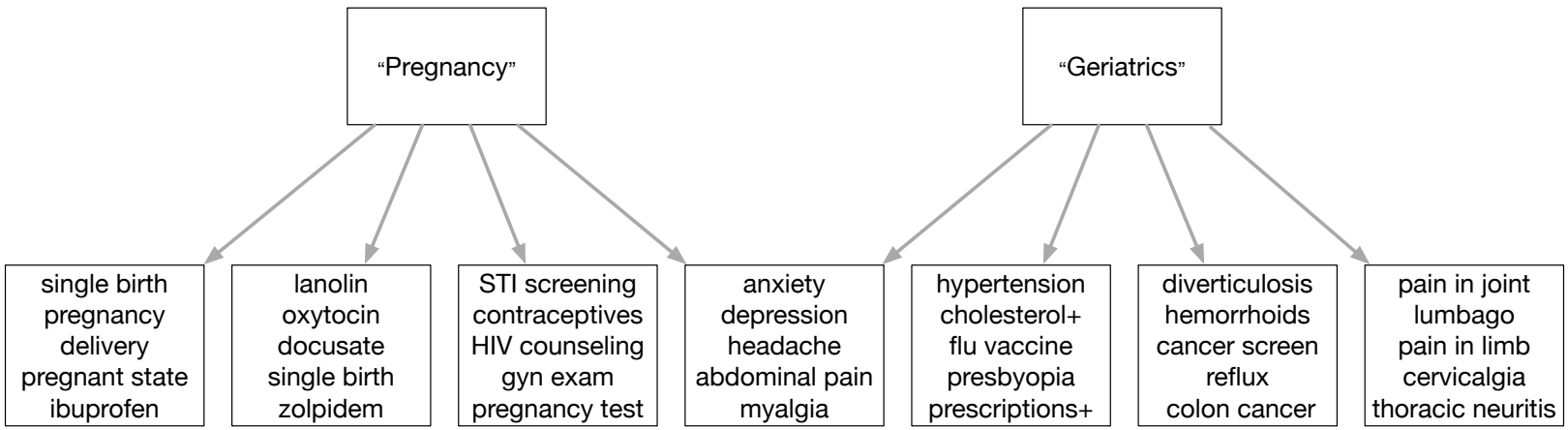
[Regier+ 2015]

Genetics



[Gopalan+ 2016]

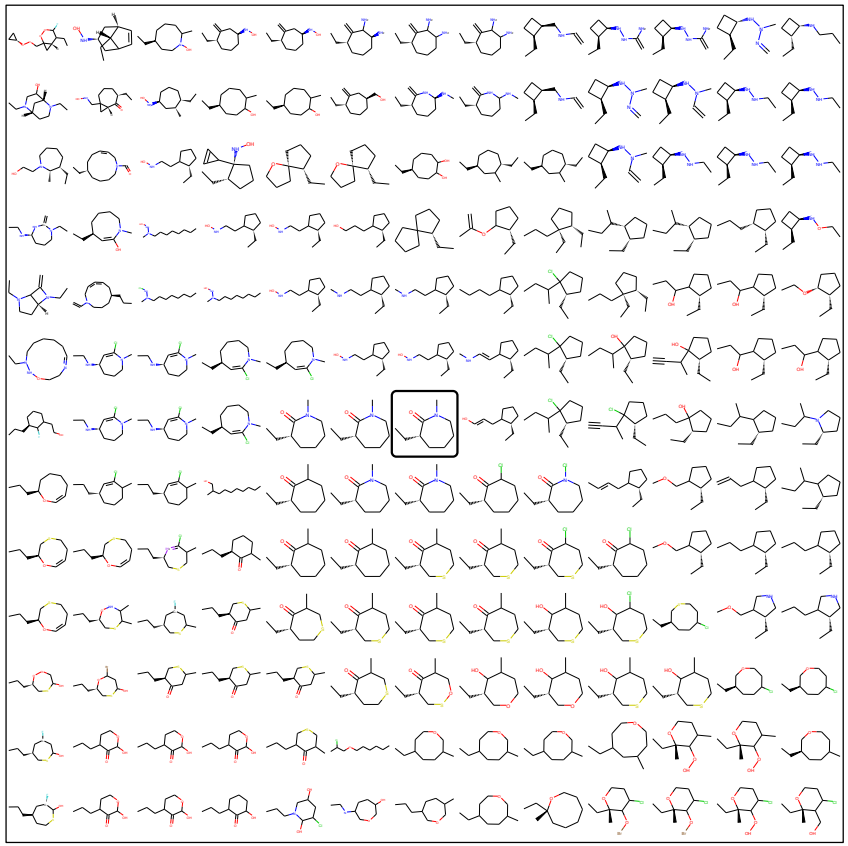
Medicine

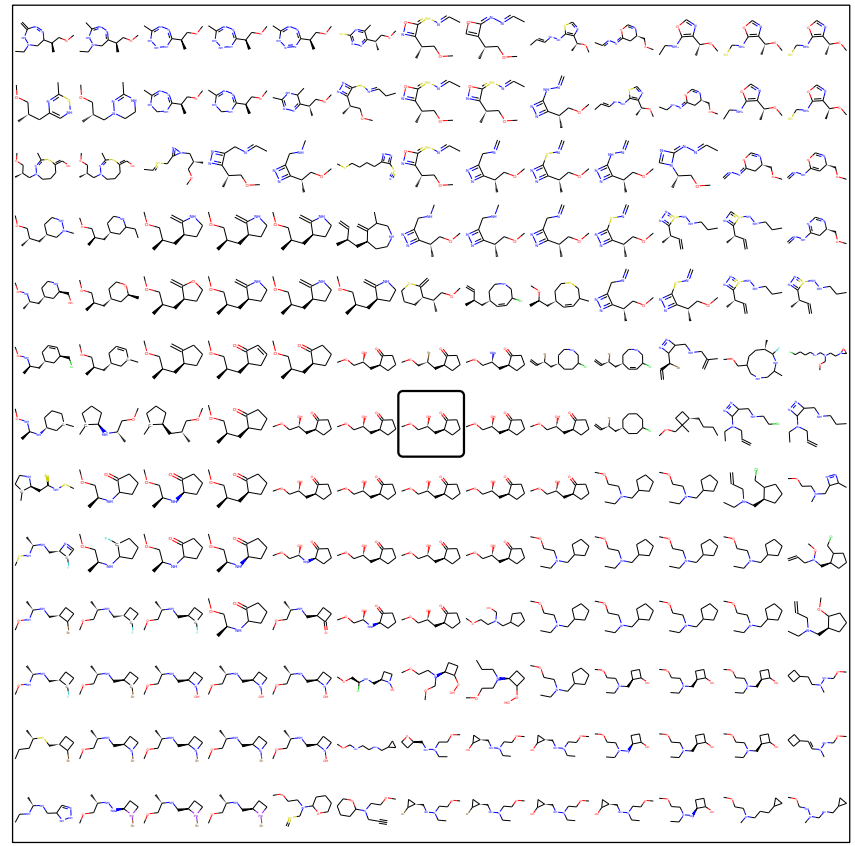

“Pregnancy”
“Geriatrics”
single birth
pregnancy
delivery
pregnant state
ibuprofen
lanolin
oxytocin
docusate
single birth
zolpidem
STI screening
contraceptives
HIV counseling
gyn exam
pregnancy test
anxiety
depression
headache
abdominal pain
myalgia
hypertension
cholesterol+
flu vaccine
presbyopia
prescriptions+
diverticulosis
hemorrhoids
cancer screen
reflux
colon cancer
pain in joint
lumbago
pain in limb
cervicalgia
thoracic neuritis

[R+]

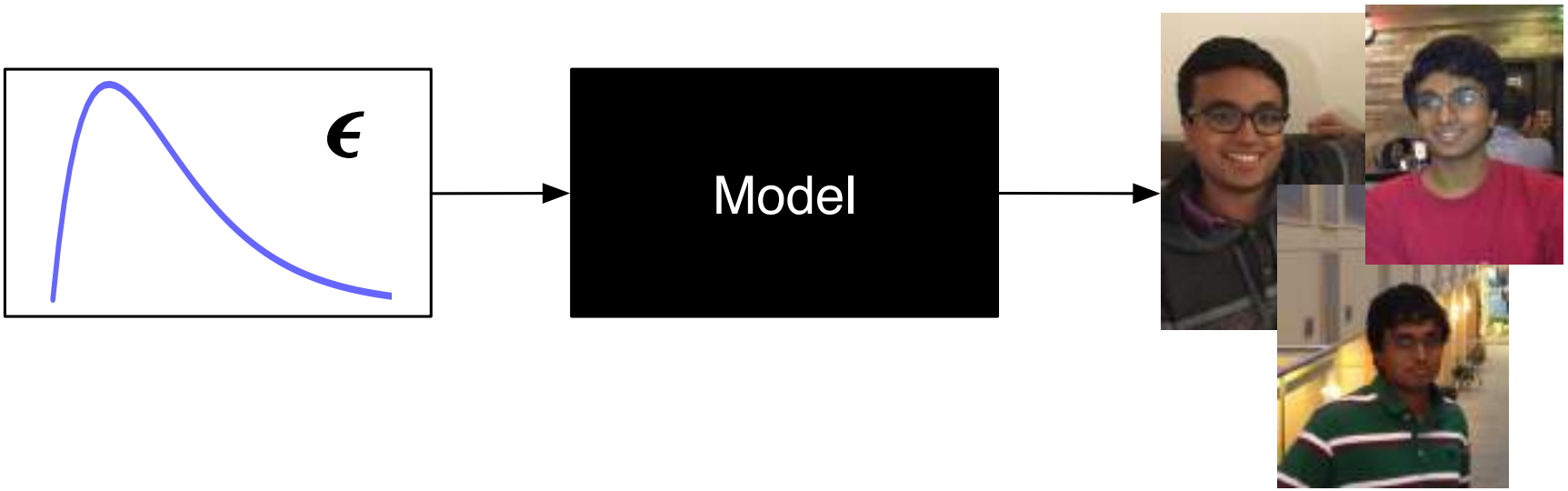
What about sampling from $p(\mathbf{x})$ directly?

Sampling Molecules





[Kusner+ 2017]

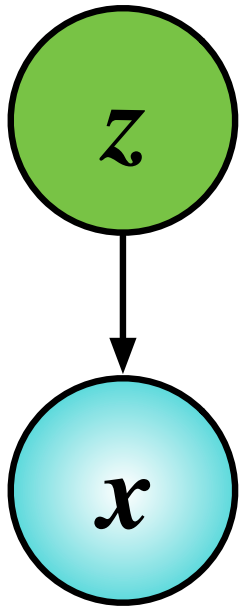

ϵ
Model

Roadmap

- **Variational Autoencoders**
- **Direct Likelihood Models**
- **Diffusion Models**

Variational Autoencoders

Variational Autoencoder (VAE)



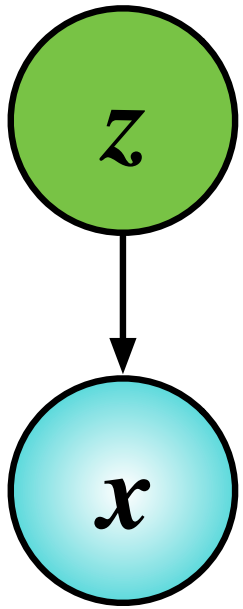
$$p(\mathbf{z}) = \text{Normal}(0, 1)$$

$$p(\mathbf{x}|\mathbf{z}) = \text{Normal}(\mu_\beta(\mathbf{z}), \sigma^2_\beta(\mathbf{z}))$$

μ and σ^2 are deep networks with parameters $\boldsymbol{\beta}$.

[Kingma+ 2014; Rezende+ 2014]

Variational Autoencoder (VAE)

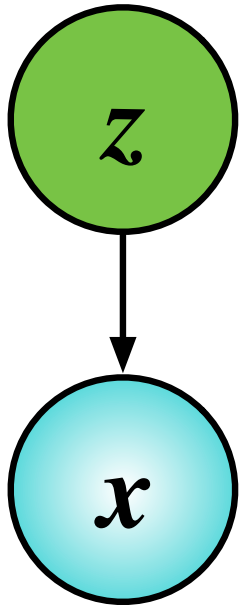


$$p(\mathbf{z}) = \text{Normal}(0, 1)$$

$$p(\mathbf{x}|\mathbf{z}) = \text{Normal}(\mu_\beta(\mathbf{z}), \sigma^2_\beta(\mathbf{z}))$$

Need to learn parameters $\boldsymbol{\beta}$ How?

Variational Autoencoder (VAE)



$$p(\mathbf{z}) = \text{Normal}(0, 1)$$

$$p(\mathbf{x}|\mathbf{z}) = \text{Normal}(\mu_\beta(\mathbf{z}), \sigma^2_\beta(\mathbf{z}))$$

Need to learn parameters $\boldsymbol{\beta}$ How? Maximum likelihood

$$\sum_{i=1}^{n} \log p_{\boldsymbol{\beta}}(\mathbf{x}_i) = \sum_{i=1}^{n} \log \int p_{\boldsymbol{\beta}}(\mathbf{x}_i, \mathbf{z}_i) d\mathbf{z}_i$$

is intractable.

Use Variational Inference!

Evidence Lower Bound (ELBO)

For a single point, would like to

$$\max_{\boldsymbol{\beta}} \log p_{\boldsymbol{\beta}}(\mathbf{x}) = \max_{\boldsymbol{\beta}} \log \int p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$

Integral is intractable

$$\log p_{\boldsymbol{\beta}}(\mathbf{x}) \quad = \quad \mathbb{E}_{q(\mathbf{z};\boldsymbol{\lambda})}[\log p_{\boldsymbol{\beta}}(\mathbf{x})] = \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})]$$

Evidence Lower Bound (ELBO)

For a single point, would like to

$$\max_{\boldsymbol{\beta}} \log p_{\boldsymbol{\beta}}(\mathbf{x}) = \max_{\boldsymbol{\beta}} \log \int p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$

Integral is intractable

$$\begin{aligned} \log p_{\boldsymbol{\beta}}(\mathbf{x}) &= \mathbb{E}_{q(\mathbf{z};\boldsymbol{\lambda})}[\log p_{\boldsymbol{\beta}}(\mathbf{x})] = \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \\ &= \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) + \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \end{aligned}$$

Evidence Lower Bound (ELBO)

For a single point, would like to

$$\max_{\boldsymbol{\beta}} \log p_{\boldsymbol{\beta}}(\mathbf{x}) = \max_{\boldsymbol{\beta}} \log \int p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$

Integral is intractable

$$\begin{aligned}
\log p_{\boldsymbol{\beta}}(\mathbf{x}) &= \mathbb{E}_{q(\mathbf{z};\boldsymbol{\lambda})}[\log p_{\boldsymbol{\beta}}(\mathbf{x})] = \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) + \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z}; \boldsymbol{\lambda})] + \mathbb{E}_q[\log q(\mathbf{z}; \boldsymbol{\lambda}) - \log p(\mathbf{z} \mid \mathbf{x})]
\end{aligned}$$

Evidence Lower Bound (ELBO)

For a single point, would like to

$$\max_{\boldsymbol{\beta}} \log p_{\boldsymbol{\beta}}(\mathbf{x}) = \max_{\boldsymbol{\beta}} \log \int p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$

Integral is intractable

$$\begin{aligned}
\log p_{\boldsymbol{\beta}}(\mathbf{x}) &= \mathbb{E}_{q(\mathbf{z};\boldsymbol{\lambda})}[\log p_{\boldsymbol{\beta}}(\mathbf{x})] = \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) + \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z}; \boldsymbol{\lambda})] + \mathbb{E}_q[\log q(\mathbf{z}; \boldsymbol{\lambda}) - \log p(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z}; \boldsymbol{\lambda})] + KL(q(\mathbf{z}; \boldsymbol{\lambda}) || p(\mathbf{z} \mid \mathbf{x}))
\end{aligned}$$

Evidence Lower Bound (ELBO)

For a single point, would like to

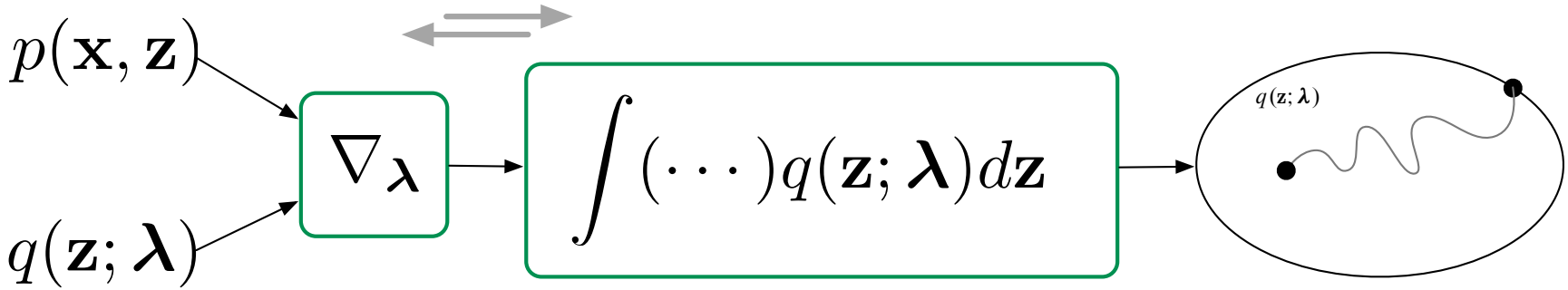
$$\max_{\boldsymbol{\beta}} \log p_{\boldsymbol{\beta}}(\mathbf{x}) = \max_{\boldsymbol{\beta}} \log \int p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) d\mathbf{z}$$

Integral is intractable

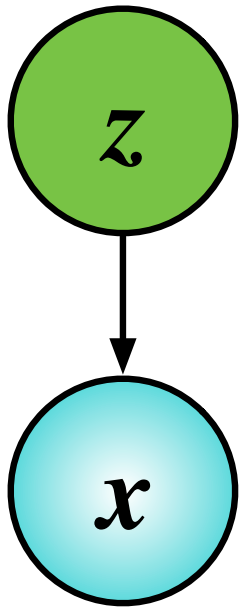
$$\begin{aligned}
\log p_{\boldsymbol{\beta}}(\mathbf{x}) &= \mathbb{E}_{q(\mathbf{z};\boldsymbol{\lambda})}[\log p_{\boldsymbol{\beta}}(\mathbf{x})] = \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p_{\boldsymbol{\beta}}(\mathbf{x}, \mathbf{z}) + \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log q(\mathbf{z}; \boldsymbol{\lambda}) - \log p_{\boldsymbol{\beta}}(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z}; \boldsymbol{\lambda})] + \mathbb{E}_q[\log q(\mathbf{z}; \boldsymbol{\lambda}) - \log p(\mathbf{z} \mid \mathbf{x})] \\
&= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z}; \boldsymbol{\lambda})] + KL(q(\mathbf{z}; \boldsymbol{\lambda}) || p(\mathbf{z} \mid \mathbf{x})) \\
&\geq \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z}; \boldsymbol{\lambda})]
\end{aligned}$$

Optimize lower bound on $\log p$ with respect to $\boldsymbol{\lambda}$ and $\boldsymbol{\beta}$

The New VI Recipe



Variational Autoencoder (VAE)



$$p(\mathbf{z}) = \text{Normal}(0, 1)$$

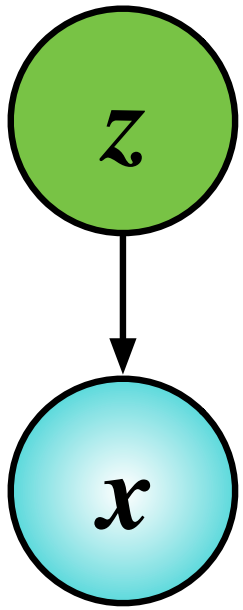
$$p(\mathbf{x}|\mathbf{z}) = \text{Normal}(\mu_\beta(\mathbf{z}), \sigma^2_\beta(\mathbf{z}))$$

Joint:

$$p(\mathbf{x}, \mathbf{z}) = \prod_{i=1}^{n} p(\mathbf{x}_i, \mathbf{z}_i) = \prod_{i=1}^{n} p(\mathbf{z}_i) p(\mathbf{x}_i \,|\, \mathbf{z}_i)$$

- $\mathbf{z}_i \sim \text{Normal}(0, 1)$
- $\mathbf{x}_i \,|\, \mathbf{z}_i \sim \text{Normal}(\mu_{\boldsymbol{\beta}}(\mathbf{z}_i), \sigma_{\boldsymbol{\beta}}(\mathbf{z}_i))$

Variational Autoencoder (VAE)



$$p(\mathbf{z}) = \text{Normal}(0, 1)$$

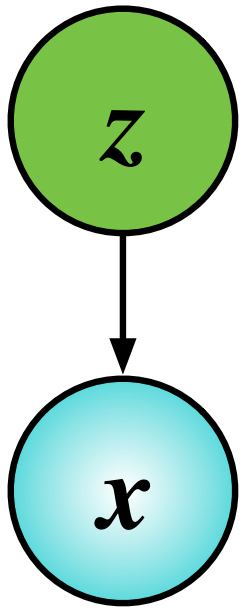
$$p(\mathbf{x}|\mathbf{z}) = \text{Normal}(\mu_\beta(\mathbf{z}), \sigma^2_\beta(\mathbf{z}))$$

Variational approximation:

$$q(\mathbf{z}) = \prod_{i=1}^{n} q(\mathbf{z}_i; \mathbf{m}_i, \mathbf{s}_i)$$

- $q(\mathbf{z}_i; \mathbf{m}_i, \mathbf{s}_i) = \text{Normal}(\mathbf{m}_i, \mathbf{s}_i)$
- Each data point has separate variational parameters

Variational Autoencoder (VAE)



$$p(\mathbf{z}) = \text{Normal}(0, 1)$$

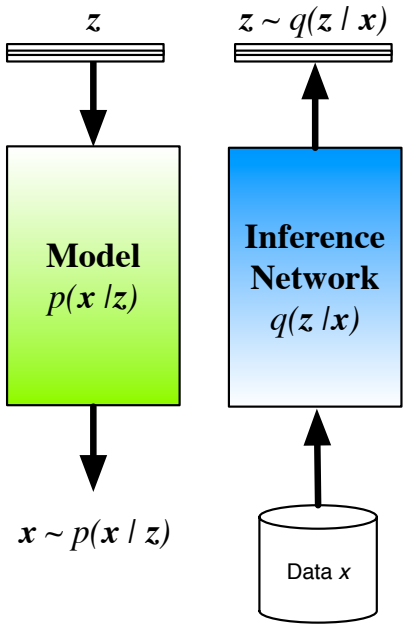
$$p(\mathbf{x}|\mathbf{z}) = \text{Normal}(\mu_\beta(\mathbf{z}), \sigma^2_\beta(\mathbf{z}))$$

Variational Objective:

$$\mathscr{L}(q, \boldsymbol{\beta}) = \mathbb{E}_q[\log p - \log q] = \sum_{i=1}^{n} \mathbb{E}_q[\log p(\mathbf{z}_i, \mathbf{x}_i) - \log q(\mathbf{z}_i; \mathbf{m}_i, \mathbf{s}_i)]$$

- Use reparameterization gradients
- Can subsample data!
- Any problems?

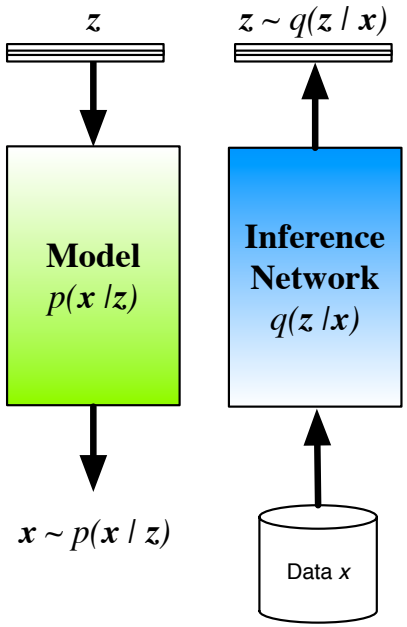
Variational Autoencoder (VAE): Inference Networks



$$q(\mathbf{z}|\mathbf{x}) = \text{Normal}(f_\theta^\mu(\mathbf{x}), f_\theta^{\sigma^2}(\mathbf{x}))$$

All functions are deep networks

Variational Autoencoder (VAE): Inference Networks



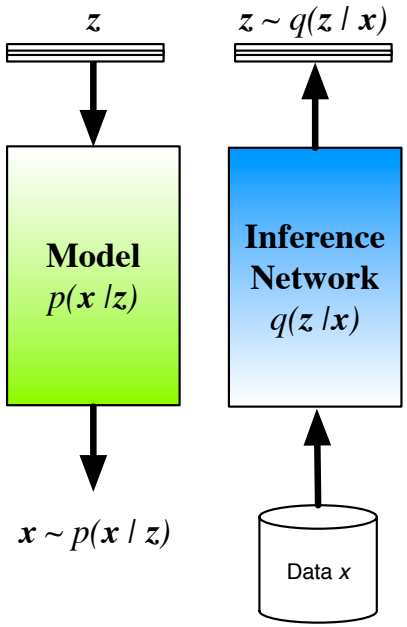
$$q(\mathbf{z}|\mathbf{x}) = \text{Normal}(f_\theta^\mu(\mathbf{x}), f_\theta^{\sigma^2}(\mathbf{x}))$$

Variational Objective:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\beta}, \boldsymbol{\theta}) &= \mathbb{E}_q[\log p - \log q] \\ &= \sum_{i=1}^{n} \mathbb{E}_q[\log p(\mathbf{z}_i, \mathbf{x}_i) - \log q(\mathbf{z}_i; \mathbf{m}_i = f_{\boldsymbol{\theta}}^\mu(\mathbf{x}_i), \mathbf{s}_i = f_{\boldsymbol{\theta}}^\sigma(\mathbf{x}_i))]\end{aligned}$$

Converts each local parameter into neural network parameters

Variational Autoencoder (VAE): Inference Networks



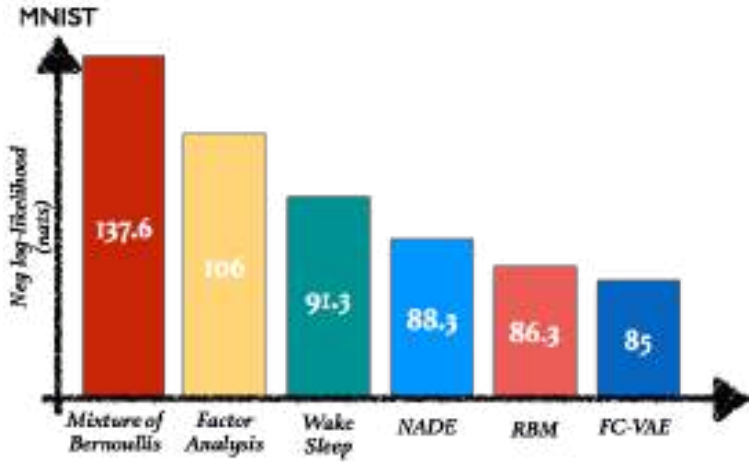
$$q(\mathbf{z}|\mathbf{x}) = \text{Normal}(f_\theta^\mu(\mathbf{x}), f_\theta^{\sigma^2}(\mathbf{x}))$$

Variational Objective:

$$\begin{aligned}\mathcal{L}(\boldsymbol{\beta}, \boldsymbol{\theta}) &= \mathbb{E}_q[\log p - \log q] \\ &= \sum_{i=1}^{n} \mathbb{E}_q[\log p(\mathbf{z}_i, \mathbf{x}_i) - \log q(\mathbf{z}_i; \mathbf{m}_i = f_{\boldsymbol{\theta}}^\mu(\mathbf{x}_i), \mathbf{s}_i = f_{\boldsymbol{\theta}}^\sigma(\mathbf{x}_i))]\end{aligned}$$

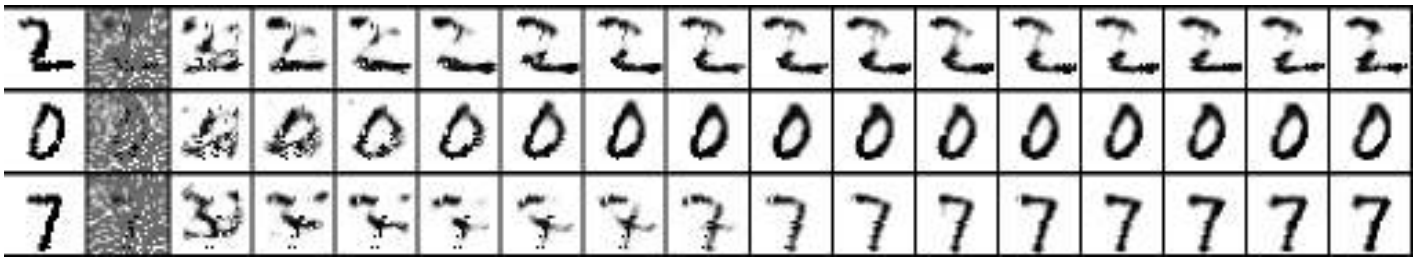
Optimize using reparameterization gradients

Variational Autoencoder (VAE)



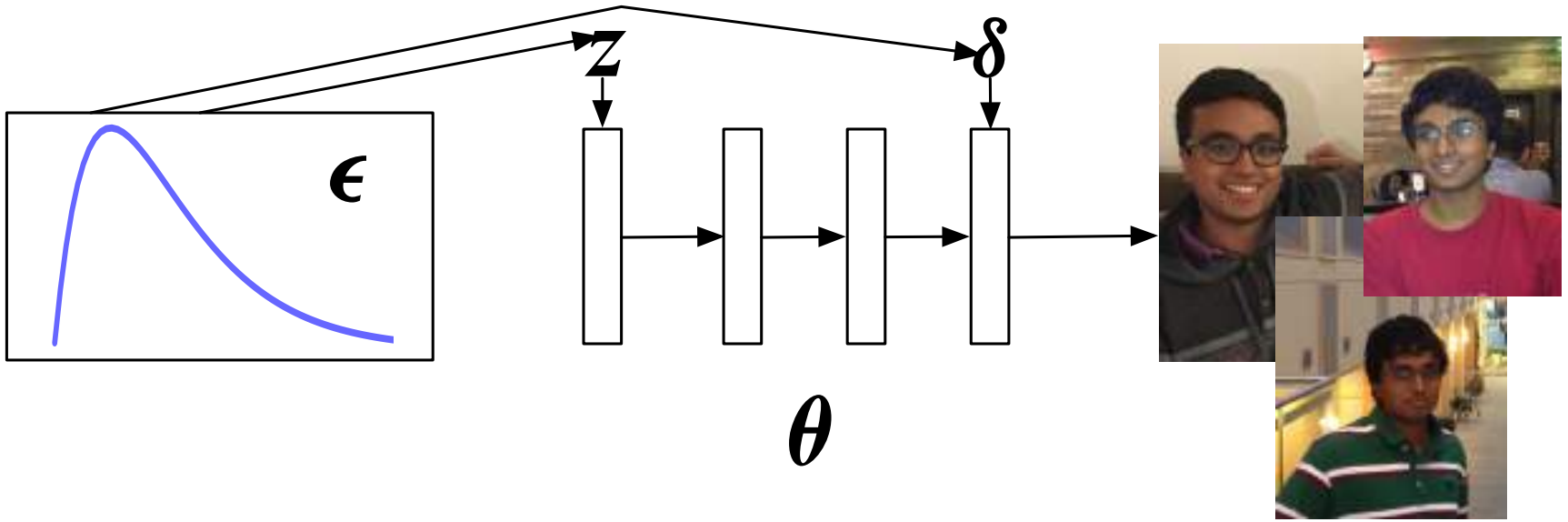






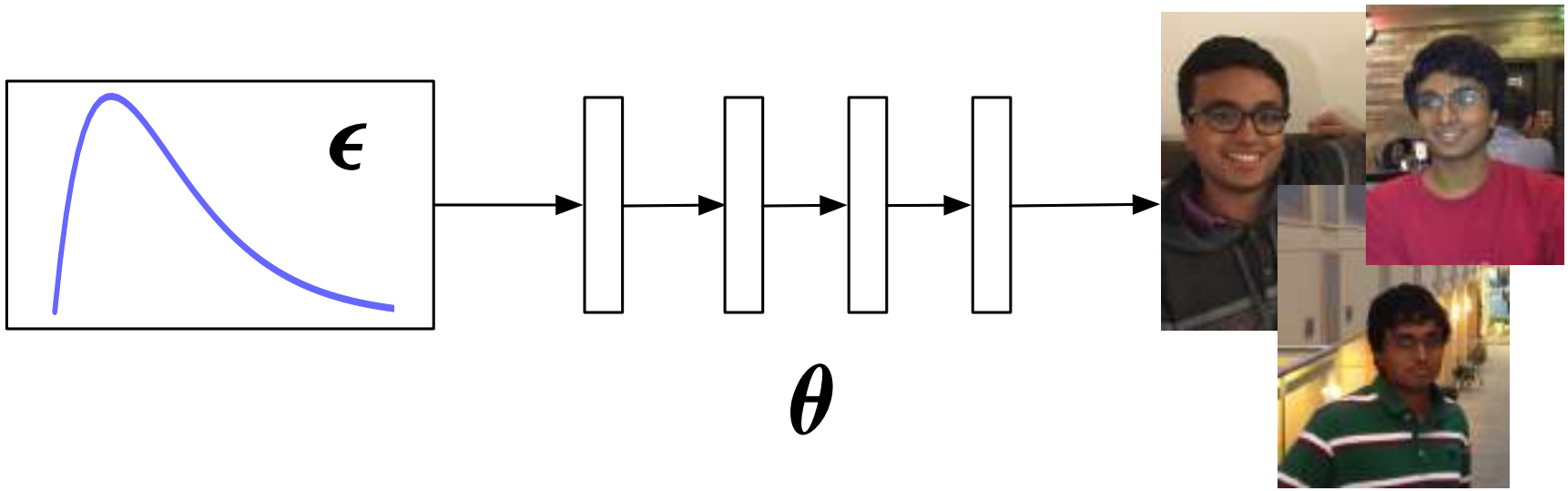
Direct Likelihood Models

Deep generative model with latent variables



- Challenge is in accurate inference of **z** given **x**
- Inference is needed because amount of noise is larger than data

Deep generative model with Noise



- Try to model density directly
- $\mathbf{x} \sim p_{\theta}(\mathbf{x}) \iff \mathbf{x} = f(\boldsymbol{\theta}, \boldsymbol{\epsilon})$

Maximum Likelihood

How do we learn a model $p_{\boldsymbol{\theta}}$ given samples $\mathbf{x}_1, ..., \mathbf{x}_n$?

1. Write down a model $p_{\boldsymbol{\theta}}(x)$
2. Write the observed likelihood of the data

$$\mathscr{L} = \log p_{\boldsymbol{\theta}}(\mathbf{x}_1, ..., \mathbf{x}_n) = \sum_{i=1}^{n} \log p_{\boldsymbol{\theta}}(\mathbf{x}_i).$$

3. Take gradient with respect to $\boldsymbol{\theta}$

$$\nabla_{\boldsymbol{\theta}} \mathscr{L} = \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathbf{x}_i).$$

4. Use (stochastic) gradients to maximize the likelihood of the data

Intuition: the model should place high probability on the data

Example: Coin Flips

Let $x_1, \ldots, x_n$ be binary observations

Use a Bernoulli model to fit $p = \sigma(\theta)$

$$\mathscr{L} = \sum_{i=1}^{n} \log p_\theta(x_i) = \sum_{i=1}^{n} x_i \theta - \log(1 + \exp(\theta))$$

Differentiate with respect to θ

$$\nabla_\theta \mathscr{L} = -n\sigma(\theta) + \sum_{i=1}^{n} x_i$$

Setting equal to zero

$$\sigma(\theta) = \frac{1}{n} \sum_{i=1}^{n} x_i$$

The maximum likelihood p equals the empirical frequency

Is Maximum Likelihood Good?

Define the empirical distribution

$$\hat{F}(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \delta_{\mathbf{x}_i}(\mathbf{x})$$

The maximum likelihood objective $\mathscr{L}$ is

$$\begin{aligned}
\mathscr{L} &= \sum_{i=1}^{n} \log p_{\boldsymbol{\theta}}(\mathbf{x}_i) \\
&= n\mathbb{E}_{\hat{F}}[\log p_{\boldsymbol{\theta}}(\mathbf{x}_i)] \\
&= n\mathbb{E}_{\hat{F}}[\log p_{\boldsymbol{\theta}}(\mathbf{x}_i) - \log \hat{F}(\mathbf{x}_i)] + C \\
&= -nKL(\hat{F}(\mathbf{x}) || p_{\boldsymbol{\theta}}(\mathbf{x})) + C
\end{aligned}$$

Maximum likelihood minimizes the KL-divergence between the empirical distribution on the data to the model

How to use maximum likelihood to fit observed data?

- We need a flexible distribution over the multivariate observations $\mathbf{x}$
- Hard for audio, text, images
- Can use known distributions like multivariate normal
- Known distributions places strong assumptions on the data

Idea 1: Decompose $p(\mathbf{x})$

Use the chain rule of probability. Let $\mathbf{x}$ be a 3-dimensional vector

$$p(\mathbf{x}) = p(\mathbf{x}_1)p(\mathbf{x}_2 \mid \mathbf{x}_1)p(\mathbf{x}_3 \mid \mathbf{x}_1, \mathbf{x}_2)$$

No assumptions were made

For a general k-dimensional vector

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_{i=1}^{k} p_{\boldsymbol{\theta}}(\mathbf{x}_i \mid \mathbf{x}_{<i})$$

Use univariate models to build a flexible multivariate model

Problem 1: Decompose p(x)

The challenge: dependencies are growing.

Consider $\mathbf{x}_3$ for $\mathbf{x}$ binary

$$p_{\boldsymbol{\theta}}(\mathbf{x}_3 \,|\, \mathbf{x}_2, \mathbf{x}_1) = \boldsymbol{\theta}[\mathbf{x}_2, \mathbf{x}_1]$$

As the dependency set grows, the conditional distribution parameter size grows exponentially.

Use deep learning to share parameters

Recurrent Neural Networks

Recurrent neural networks are sequence models

$$h_i = f_{\boldsymbol{\theta}}(h_{i-1}, \mathbf{x}_{i-1})$$

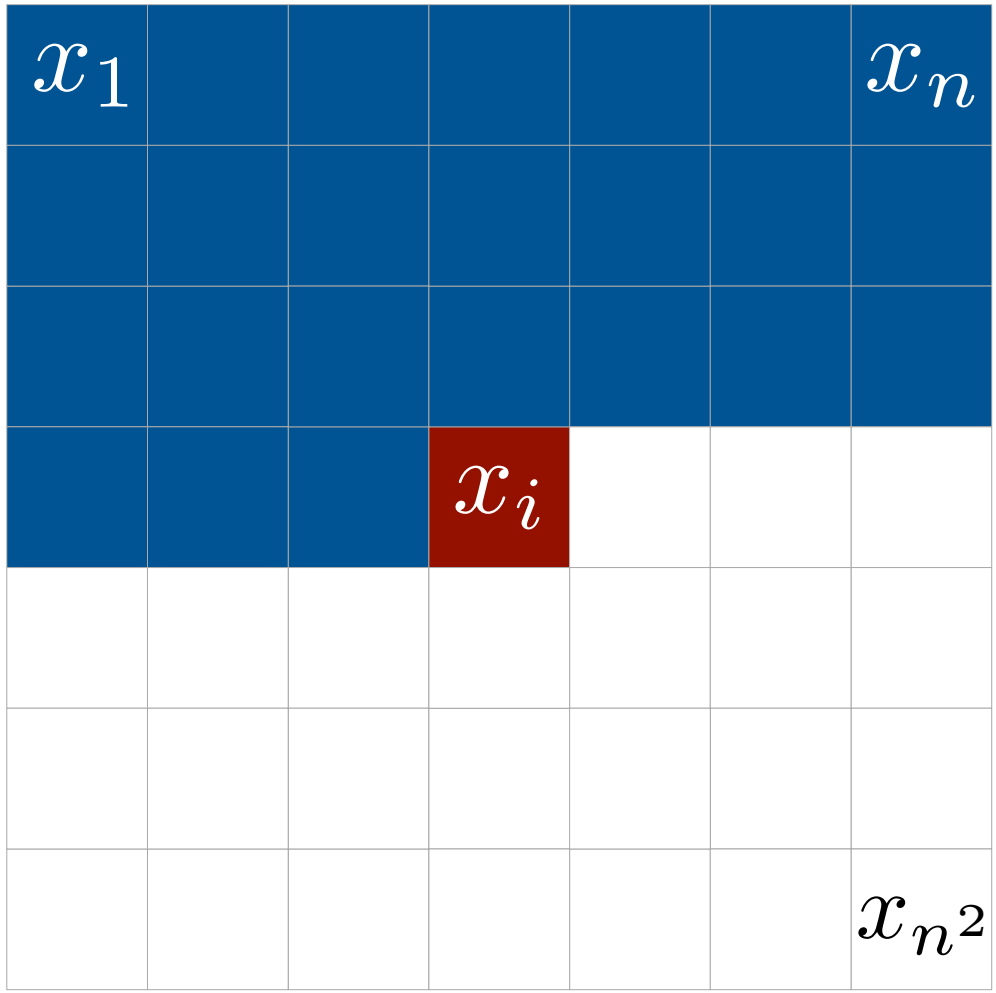
The hidden state h_i has information about all the previous $\mathbf{x}_{<i}$.

Use this to build a probability model over $\mathbf{x}$

$$\begin{aligned} h_i &= f_{\boldsymbol{\theta}}(h_{i-1}, \mathbf{x}_{i-1}) \\ p_{\boldsymbol{\theta}}(\mathbf{x}_i \mid \mathbf{x}_{<i}) &= p(\mathbf{x}_i \mid h_i) \end{aligned}$$

Shares parameters among the univariate density problems

Still a Problem?



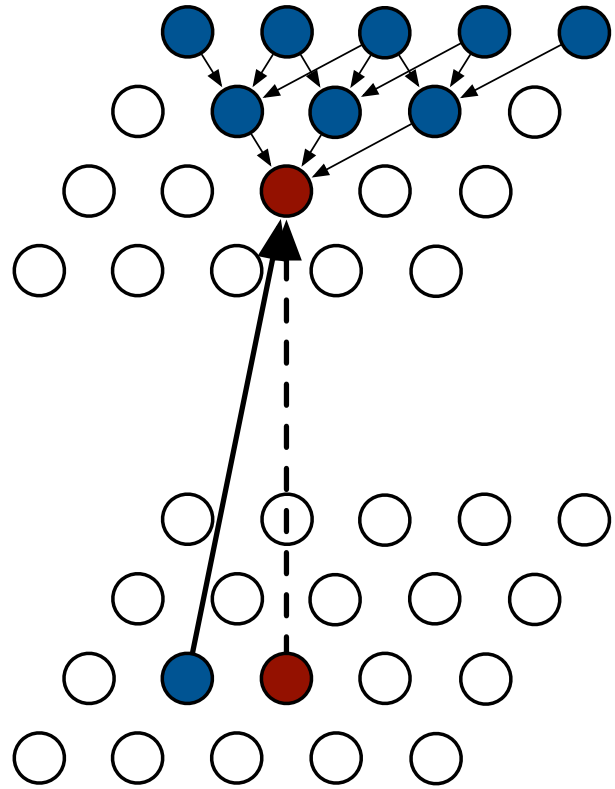
Long range dependencies are hard. Related pixels are far!

[van den Oord+ 2016]

Row LSTM

Operate on rows instead of pixels

$$
\begin{aligned}
[\mathbf{o}_i, \mathbf{f}_i, \mathbf{i}_i, \mathbf{g}_i] &= \sigma(\mathbf{K}^{ss} \circledast \mathbf{h}_{i-1} + \mathbf{K}^{is} \circledast \mathbf{x}_i) \\
\mathbf{c}_i &= \mathbf{f}_i \odot \mathbf{c}_{i-1} + \mathbf{i}_i \odot \mathbf{g}_i \\
\mathbf{h}_i &= \mathbf{o}_i \odot \tanh(\mathbf{c}_i)
\end{aligned}
$$

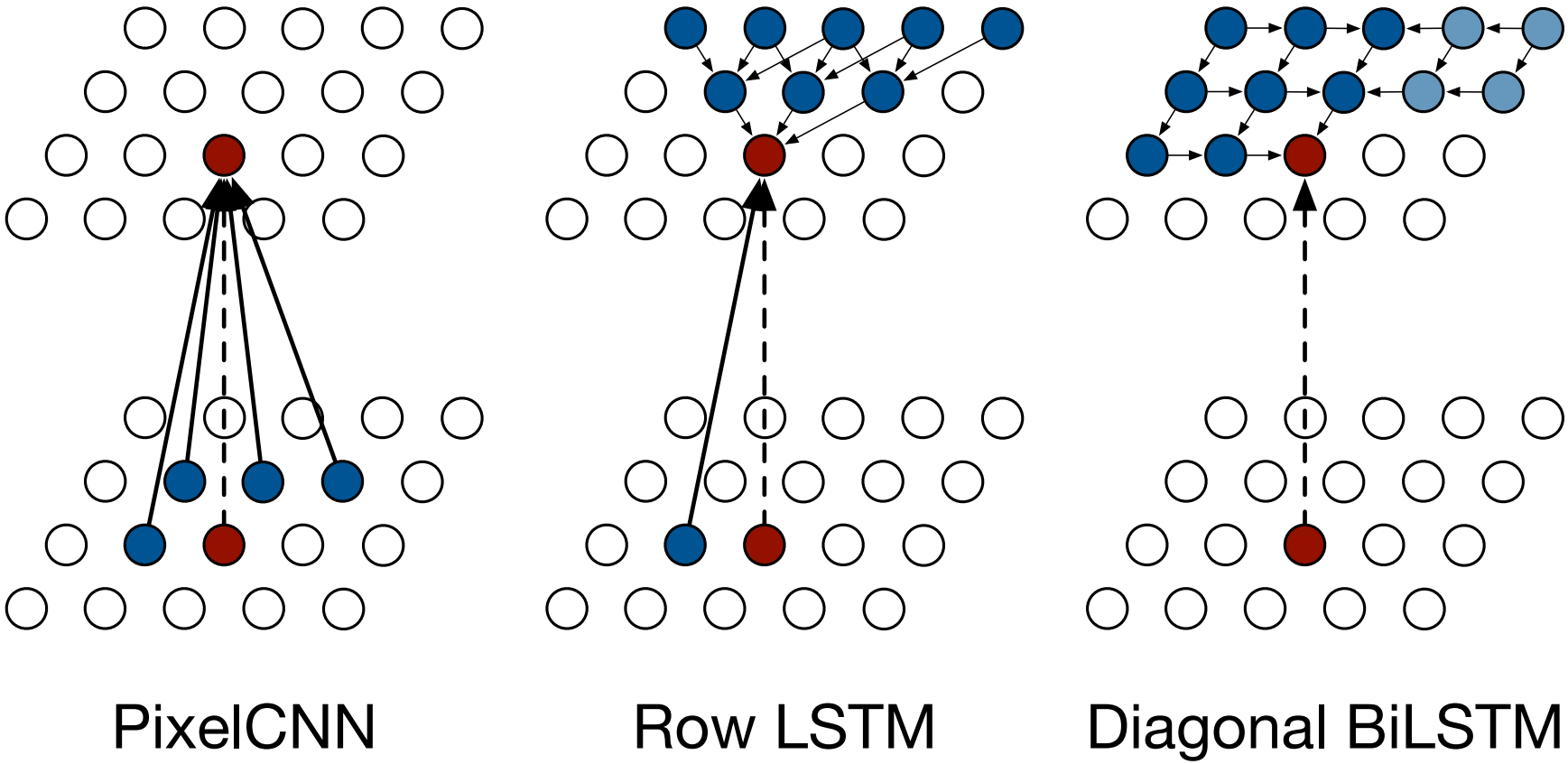


Row LSTM

Even more parameter sharing with convolution and stacking

[van den Oord+ 2016]

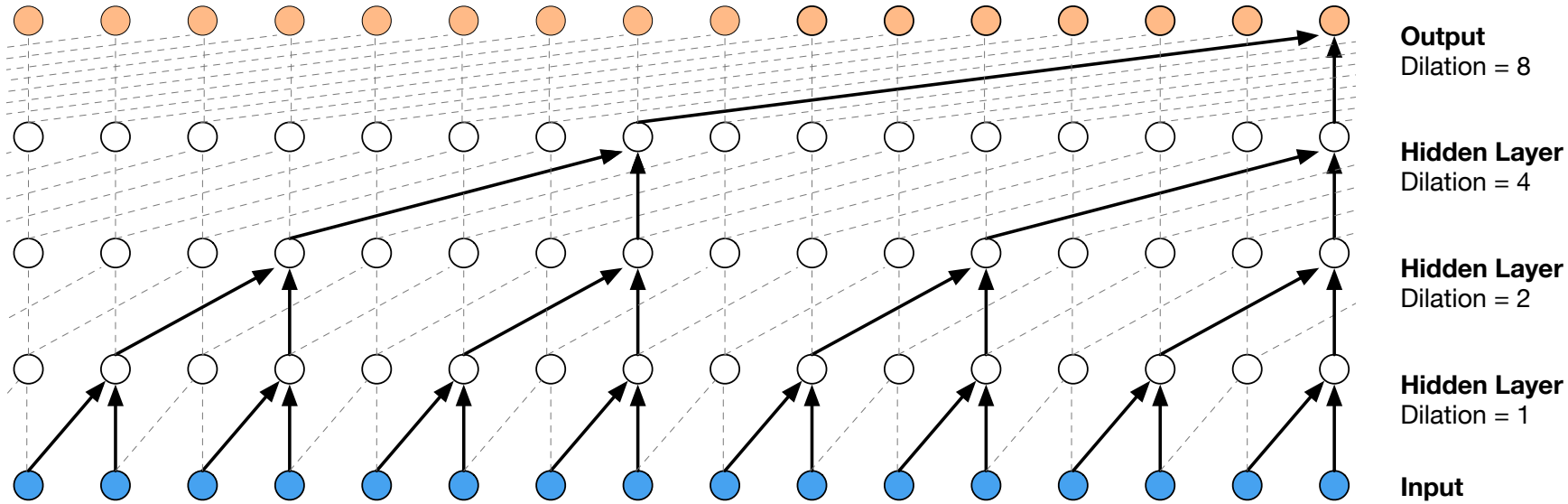
Different Architectures



Tradeoff receptive field size, computation, and flexibility

Different Architectures

Get dependencies by convolution with spacing



- Exponential spacing means exponentially long context
- Originally took 90 minutes to generate one second
- In 2017 reported a 1,000 faster approach. Now in Google Assistant

[van den Oord+ 2016]

Results



[van den Oord+ 2016]

Idea 2: Transform Noise Vectors

Directly get k dimensional densities by transforming noise

$$\boldsymbol{\epsilon} \sim s(\boldsymbol{\epsilon})$$
$$\mathbf{x} = f_{\boldsymbol{\theta}}(\boldsymbol{\epsilon})$$

What's the density of $\mathbf{x}$? Assume f is invertible

$$p(\mathbf{x}) = s(f_{\boldsymbol{\theta}}^{-1}(\mathbf{x})) \left| \frac{df_{\boldsymbol{\theta}}^{-1}(\mathbf{x})}{d\mathbf{x}} \right|$$

Easy to write!

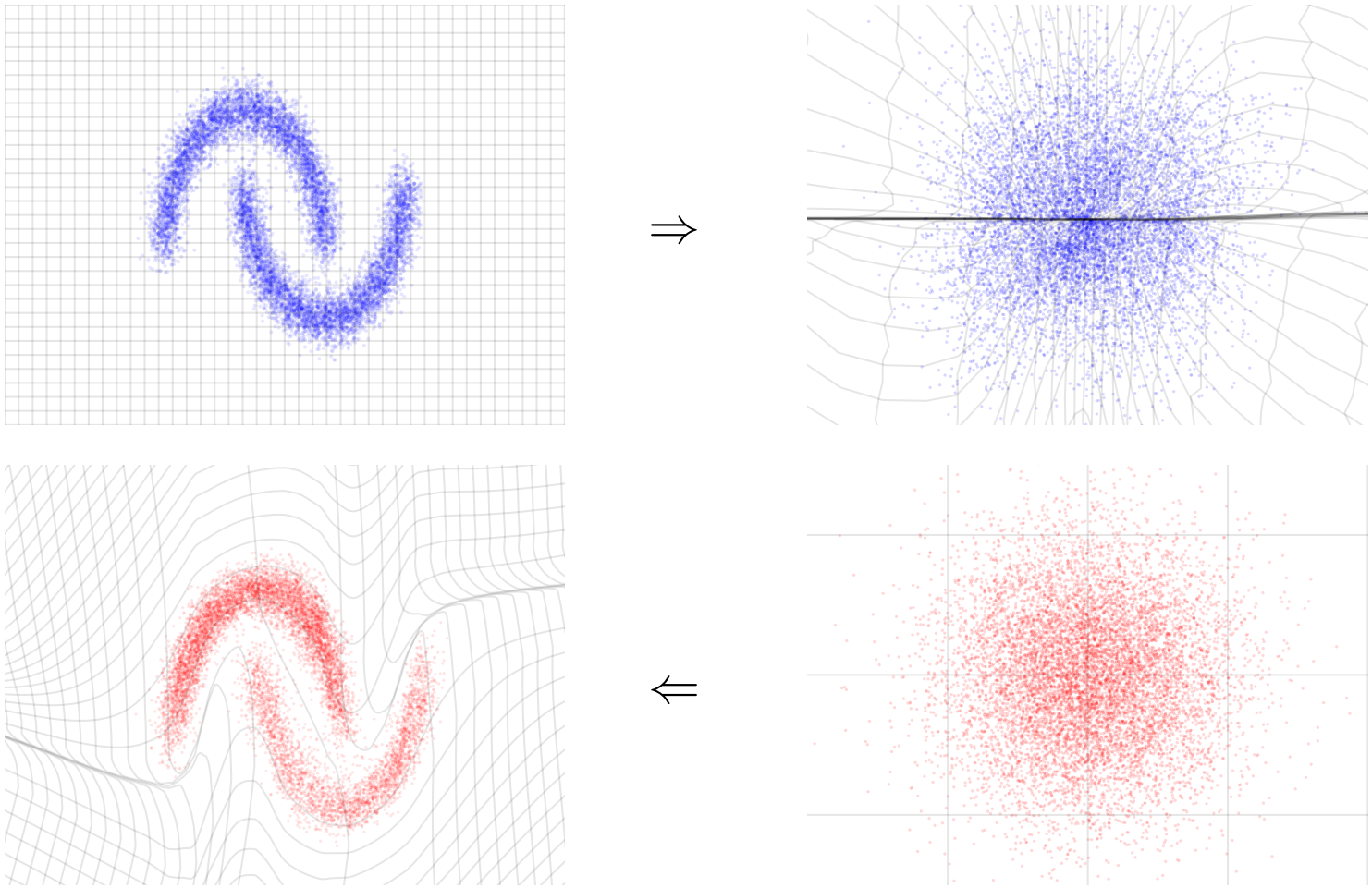
Problem 2: Jacobians are slow

Density of $\mathbf{x}$

$$p(\mathbf{x}) = s(f_{\boldsymbol{\theta}}^{-1}(\mathbf{x})) \left| \frac{df_{\boldsymbol{\theta}}^{-1}(\mathbf{x})}{d\mathbf{x}} \right|$$

- Requires computing the determinant of a $k \times k$ matrix
- Determinant computation in general is cubic
- Sampling and density requires both f and its inverse to be easy to compute

$$h(x) = x^x, \quad x > 1$$



[Dinh+ 2017]

Solution 2: Use functions O(n) Jacobians

Lower triangular matrix determinant can be computed in O(n)

$$|A| = \prod_{i=1}^{k} |a_{ii}|,$$

How do you build a function with lower triangular Jacobian? Limit the dependencies between dimensions

$$\begin{aligned}
\mathbf{x}_1 &= f_{\boldsymbol{\theta}}(\boldsymbol{\epsilon}_1) \\
\mathbf{x}_2 &= f_{\boldsymbol{\theta}}(\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2) \\
\mathbf{x}_3 &= f_{\boldsymbol{\theta}}(\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2, \boldsymbol{\epsilon}_3)
\end{aligned}$$

We have

$$\frac{d\mathbf{x}_1}{d\boldsymbol{\epsilon}_{>1}} = 0$$

Example: Real NVP

Transformation:

$$\mathbf{x}_{1:d} = \boldsymbol{\epsilon}_{1:d}$$
$$\mathbf{x}_{d+1:k} = \boldsymbol{\epsilon}_{d+1:k} \cdot \exp(s(\boldsymbol{\epsilon}_{1:d})) + t(\boldsymbol{\epsilon}_{1:d})$$

Jacobian lower triangular because $\mathbf{x}_d$ depends only on $\boldsymbol{\epsilon}_{\leq d}$
Can compose functions because

$$|J[f \circ g]| = |Jf||Jg|$$

Tradeoff in choosing d:

- Large d means transformation functions s and t depend on large part of the vector
- Small d means more variables get transformed

[Dinh+ 2017]

Example: Real NVP

Transformation:

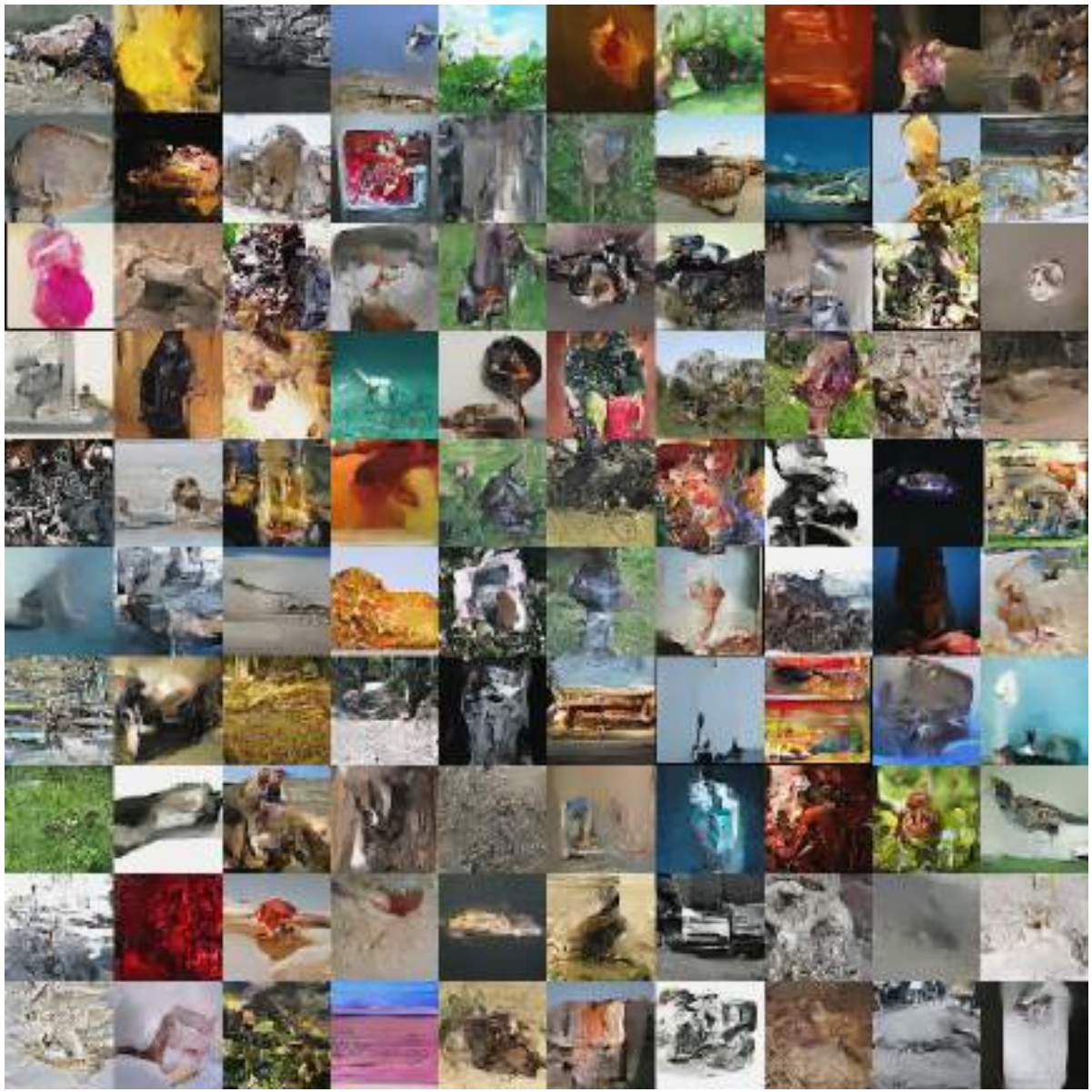
$$\mathbf{x}_{1:d} = \boldsymbol{\epsilon}_{1:d}$$
$$\mathbf{x}_{d+1:k} = \boldsymbol{\epsilon}_{d+1:k} \cdot \exp(s(\boldsymbol{\epsilon}_{1:d})) + t(\boldsymbol{\epsilon}_{1:d})$$

Jacobian lower triangular because $\mathbf{x}_d$ depends only on $\boldsymbol{\epsilon}_{\leq d}$

- Functions can use convolution or other prior knowledge
- The d can change so all dimensions can effect other dimensions with enough composition

[Dinh+ 2017]

Real NVP in Action



[Dinh+ 2017]

A Connection

Consider the autoregressive case where $\mathbf{x}$ is three dimensional

$$p(\mathbf{x}) = p_{\theta}(\mathbf{x}_1)p_{\theta}(\mathbf{x}_2 \mid \mathbf{x}_1)p_{\theta}(\mathbf{x}_3 \mid \mathbf{x}_1, \mathbf{x}_2)$$

Implies for some noise $\boldsymbol{\epsilon}$

$$\mathbf{x}_1 = f_{\theta}(\boldsymbol{\epsilon}_1), \quad \mathbf{x}_2 = f_{\theta}(\boldsymbol{\epsilon}_2, \mathbf{x}_1), \quad \mathbf{x}_3 = f_{\theta}(\boldsymbol{\epsilon}_3, \mathbf{x}_1, \mathbf{x}_2)$$

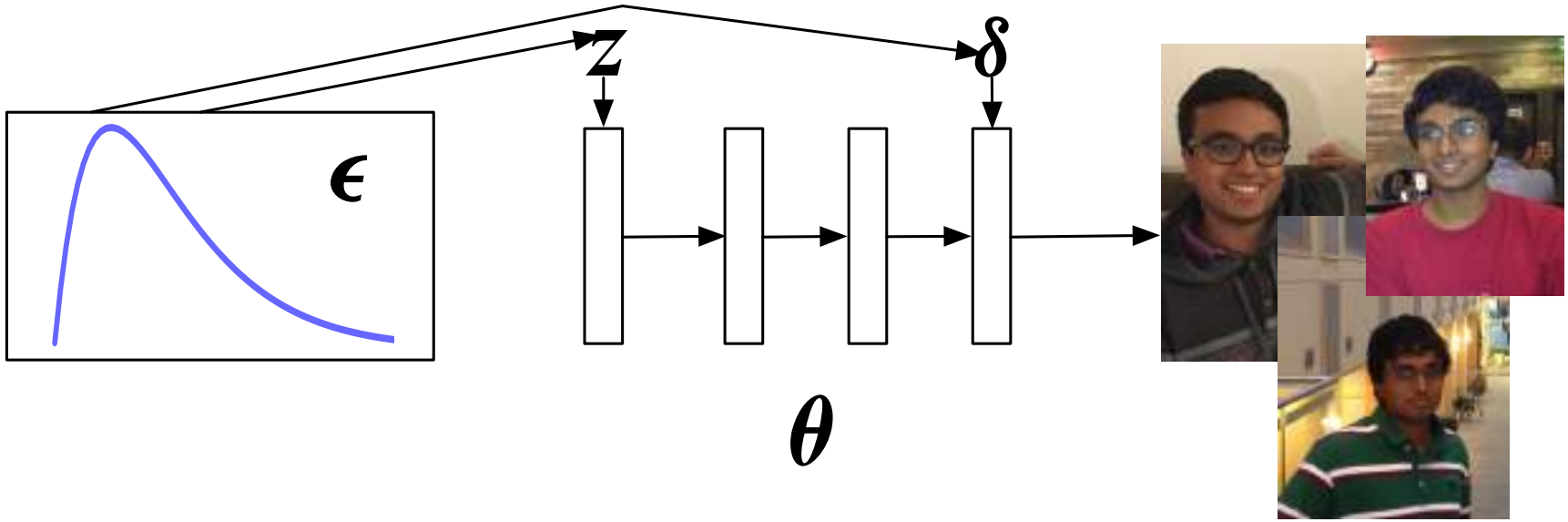
Substituting definitions of $\mathbf{x}_1$ and $\mathbf{x}_2$

$$\begin{aligned}
\mathbf{x}_1 &= f_{\theta}(\boldsymbol{\epsilon}_1) \\
\mathbf{x}_2 &= f_{\theta}(\boldsymbol{\epsilon}_2, f_{\theta}(\boldsymbol{\epsilon}_1)) \\
\mathbf{x}_3 &= f_{\theta}(\boldsymbol{\epsilon}_3, f_{\theta}(\boldsymbol{\epsilon}_1), f_{\theta}(\boldsymbol{\epsilon}_2, f_{\theta}(\boldsymbol{\epsilon}_1)))
\end{aligned}$$

This gives a vector transform with lower triangular Jacobian. The two approaches are almost the same!

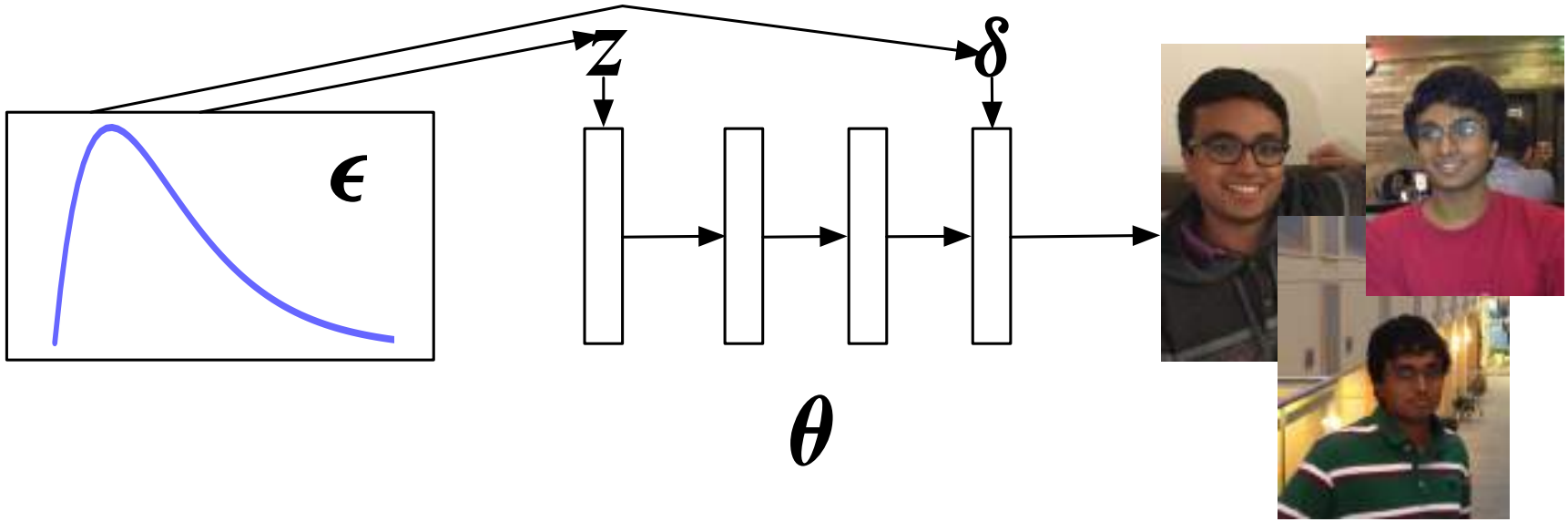
Diffusion Models

Deep generative model with latent variables



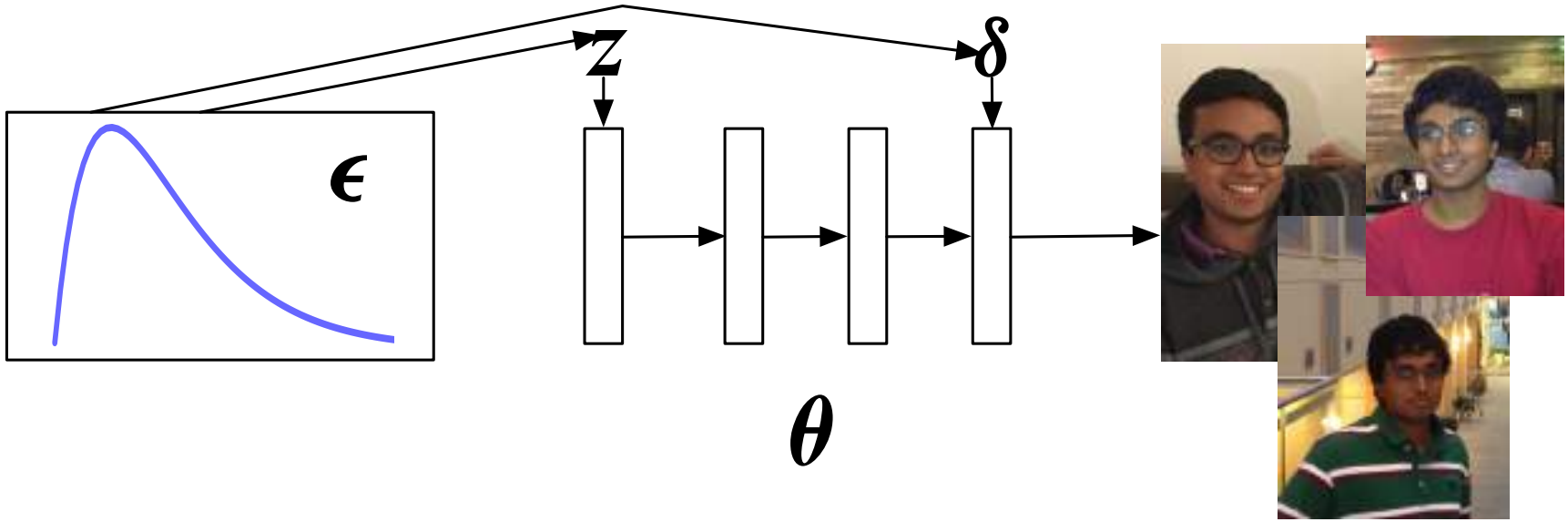
Build deep generative model using layers of hidden variables **z**

Deep generative model with latent variables



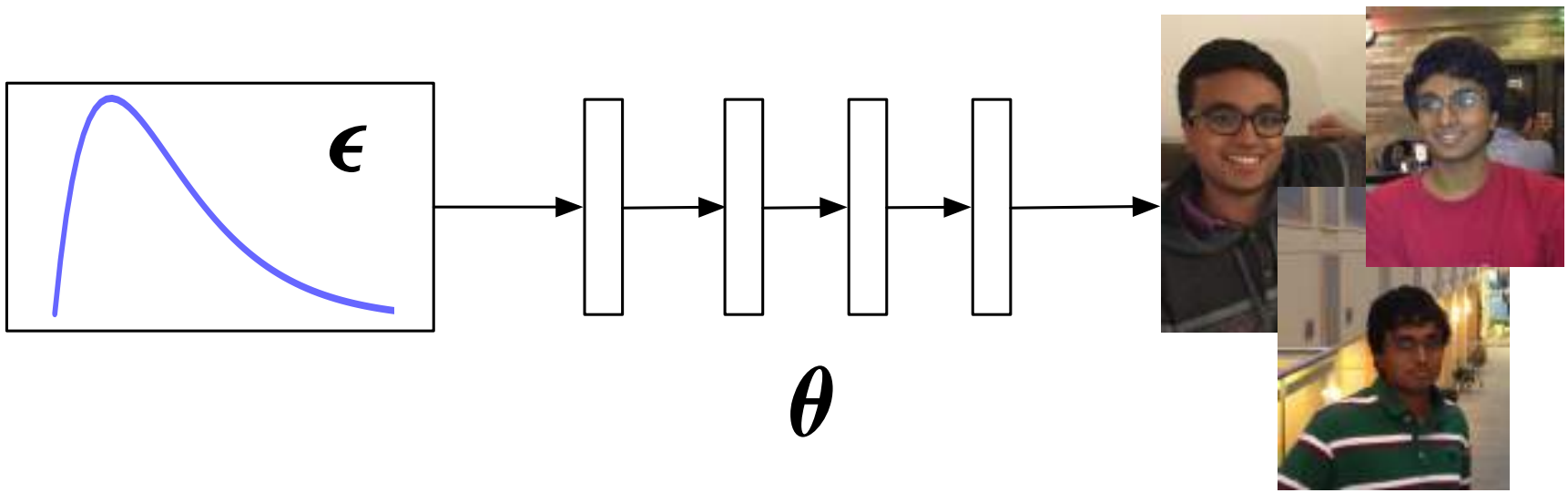
- Hidden variables have prior $p(\mathbf{z})$
- Data conditions on the hidden variables $p(\mathbf{x}|\mathbf{z})$

Deep generative model with latent variables



- Challenge is in accurate inference of **z** given **x**
- Inference is needed because amount of noise is larger than data

Deep generative model with Noise



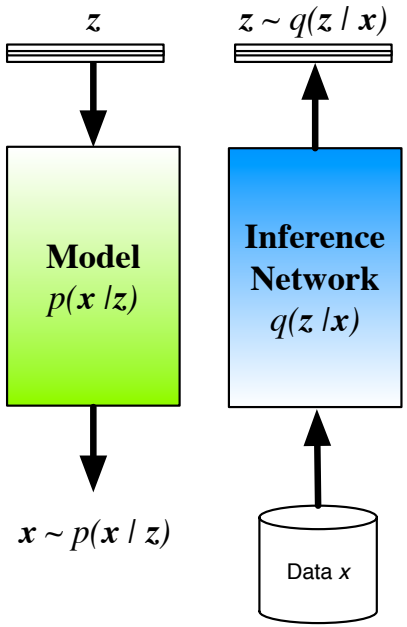
- Try to model density directly
- $\mathbf{x} \sim p_{\theta}(\mathbf{x}) \iff \mathbf{x} = f(\boldsymbol{\theta}, \boldsymbol{\epsilon})$
- f has to be invertible for easy density

Deep generative model with Noise

Options

- Inference is not too hard
- Can use arbitrary neural networks
- Could use GANs, but all parameters do not follow the same objective (maybe more on GANs at the end of the class)

The challenge



$$q(\mathbf{z}|\mathbf{x}) = \text{Normal}(f_\theta^\mu(\mathbf{x}), f_\theta^{\sigma^2}(\mathbf{x}))$$

Requires

- Fitting a model p
- Having good inference q for that p

Inference has to be accurate and "keep track" of p

Idea: Fix inference and learn the model?

Expected ELBO over data $q(\mathbf{x})$:

$$\begin{aligned}
\mathscr{L} &= \mathbb{E}_{q(\mathbf{x})}\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x},\mathbf{z}) - \log q(\mathbf{z}|\mathbf{x})] \\
&= \mathbb{E}_{q(\mathbf{x})}\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x},\mathbf{z}) - \log q(\mathbf{x},\mathbf{z}) + \log q(\mathbf{x})] \\
&= \mathbb{E}_{q(\mathbf{x})}\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z}) + \log p(\mathbf{x}|\mathbf{z}) - \log q(\mathbf{z}) - \log q(\mathbf{x}|\mathbf{z}) + \log q(\mathbf{x})] \\
&= -\mathrm{KL}(q(\mathbf{z})||p(\mathbf{z})) - \mathbb{E}_{q(\mathbf{z})}[\mathrm{KL}(q(\mathbf{x}|\mathbf{z})||p(\mathbf{x}|\mathbf{z}))] + \mathbb{E}_{q(\mathbf{x})}[\log q(\mathbf{x})]
\end{aligned}$$

$\mathrm{KL}(q(\mathbf{z})||p(\mathbf{z}))$ needs to be small for large ELBO

How can can we pick

- The inference $q_\lambda(\mathbf{x}|\mathbf{z})$ is fixed
- $\mathbb{E}_{q(\mathbf{x})}[q_\lambda(\mathbf{z}|\mathbf{x})] \approx p(\mathbf{z})$ for any data distribution $q(\mathbf{x})$

Crash Course: Stochastic Differential Equations

Stochastic differential equation

$$d\mathbf{z}_t = f(\mathbf{z}_t, t)dt + g(\mathbf{z}_t, t)dB_t$$
$$\mathbf{z}_0 = c$$

Intuition from simulation to a time T:

1. Initialize $\mathbf{z}_0 = c$, $t = 0$
2. While $t < T$
   - $\mathbf{z}_{t+\epsilon} = f(\mathbf{z}_t, t)\epsilon + g(\mathbf{z}_t, t)\text{Normal}(0, \text{variance} = \epsilon)$
   - $t = t + \epsilon$

SDEs start from an initial condition and add in a deterministic drift and Gaussian noise

Idea: Fix inference and learn the model!

Consider a stochastic process

$$d\mathbf{z}_t = -\frac{1}{2}\sigma(t)\mathbf{z}_t dt + \sqrt{\sigma(t)}dB_t$$
$$\mathbf{z}_0 = \mathbf{x}$$

For any $\mathbf{x}$, this stochastic process converges in distribution to

$$\text{Normal}(0, I)$$

This provides a way

- The inference $q_\lambda(\mathbf{x}|\mathbf{z})$ is fixed
- $\mathbb{E}_{q(\mathbf{x})}[q_\lambda(\mathbf{z}|\mathbf{x})] \approx p(\mathbf{z})$ for any data distribution $q(\mathbf{x})$

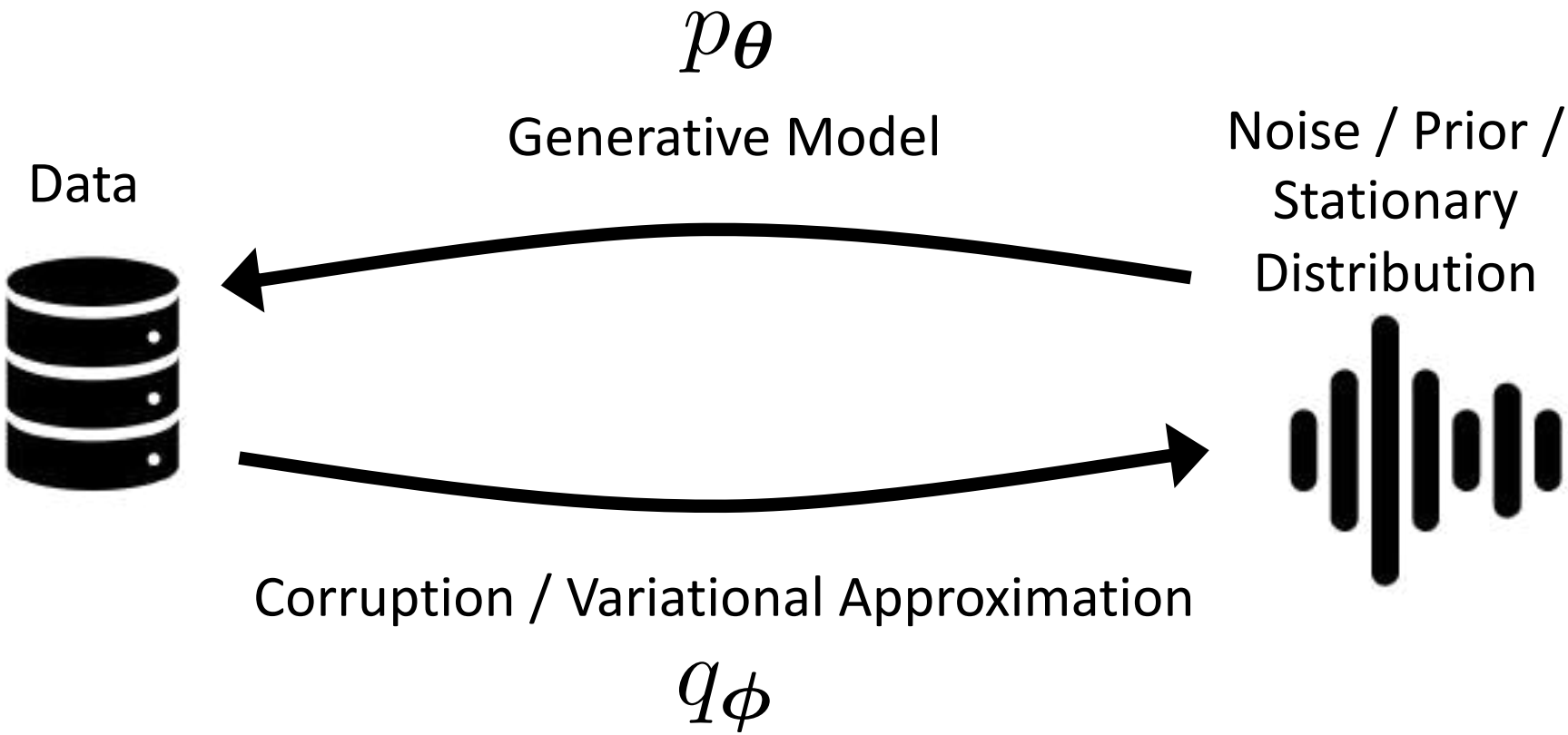
ELBO (without KL)

$$
\begin{aligned}
\log p(\mathbf{x}) &= \log \mathbb{E}_{p(\mathbf{z})}[p(\mathbf{x}\,|\,\mathbf{z})] && \text{Marginal likelihood} \\
&= \log \mathbb{E}_{q(\mathbf{z}\,|\,\mathbf{x})}\left[\frac{p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})} p(\mathbf{x}\,|\,\mathbf{z})\right] && \text{Introduce inference} \\
&\geq \mathbb{E}_{q(\mathbf{z}\,|\,\mathbf{x})}\left[\log\left(\frac{p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})} p(\mathbf{x}\,|\,\mathbf{z})\right)\right] && \text{Use Jensens} \\
&= \mathbb{E}_{q(\mathbf{z}\,|\,\mathbf{x})}\left[\log p(\mathbf{z}) - \log q(\mathbf{z}\,|\,\mathbf{x}) + \log p(\mathbf{x}\,|\,\mathbf{z})\right] && \text{Simplify}
\end{aligned}
$$

Steps to A Diffusion ELBO

- Define a latent variable model with an SDE
- Write the marginal likelihood
- Rewrite the marginal likelihood using inference process q
- Use Jensen's inequality
- Simplify

High Level View



A latent variable model

Model distribution:

$$\begin{aligned}
\mathbf{a}_0 &\sim p_0(\mathbf{a}_0) \\
d\mathbf{a}_t &= h(\mathbf{a}_t, t)dt + \delta(t)dB_t \\
\mathbf{z}_t &= \mathbf{a}_{1-t} \\
\mathbf{x} &\sim p(\mathbf{x} \mid \mathbf{z}_\epsilon)
\end{aligned}$$

- The function h is vector valued
- The process $\mathbf{z}$ is the latent variable
- The value $\mathbf{x}$ is the observation or data

Need to compute $\log p(\mathbf{z}_\epsilon)$

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})}[\log p(\mathbf{x} \mid \mathbf{z}_\epsilon) + \log p(\mathbf{z}_\epsilon) - \log q(\mathbf{z}_\epsilon \mid \mathbf{x})]$$

Fokker-Planck: Marginal Likelihood 1

Model distribution:

$$\begin{aligned}\mathbf{a}_0 &\sim p_0(\mathbf{a}_0)\\ d\mathbf{a}_t &= h(\mathbf{a}_t, t)dt + \delta(t)dB_t\\ \mathbf{z}_t &= \mathbf{a}_{1-t}\end{aligned}$$

Recall the divergence of a field

$$\nabla \cdot f(\mathbf{x}) = \sum_{i=1}^{D} \frac{d}{d\mathbf{x}_i} f_i(\mathbf{x}_i)$$

The time evolution of the density of an SDE is

$$\begin{aligned}\partial_t p(\mathbf{a}, t) &= -\nabla \cdot [h(\mathbf{a}, t)p(\mathbf{a}, t)] + \delta(t)\nabla \cdot \nabla p(\mathbf{a}, t)\\ p(\mathbf{a}, 0) &= p(\mathbf{a}_0)\end{aligned}$$

The density $p(\mathbf{z}, \boldsymbol{\epsilon}) = p(\mathbf{a}, 1 - \boldsymbol{\epsilon})$ is the solution to a PDE

Feynman-Kac: Marginal Likelihood 2

The solution to the PDE

$$\partial_t p(\mathbf{a}, t) = -\nabla \cdot [h(\mathbf{a}, t) p(\mathbf{a}, t)] + \delta(t) \nabla \cdot \nabla p(\mathbf{a}, t)$$
$$p(\mathbf{a}, 0) = p_0(\mathbf{a}_0)$$

can be written as an SDE

$$p(\mathbf{a}, 1-\epsilon) = \mathbb{E}\left[p_0(\mathbf{z}_1) \exp\left(\int_{\epsilon}^{1} -\nabla \cdot h(\mathbf{z}_s, 1-s) ds \right) | \, \mathbf{z}_\epsilon = \mathbf{a} \right]$$
$$d\mathbf{z}_t = -h(\mathbf{z_t}, 1-t)dt + \sigma(1-t) dB_t$$

The density can be written as an expectation!

ELBO (without KL)

$$\log p(\mathbf{x}) = \log \mathbb{E}_{p(\mathbf{z})}[p(\mathbf{x}|\mathbf{z})] \qquad \text{Marginal likelihood}$$

$$= \log \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\left[\frac{p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x})} p(\mathbf{x}|\mathbf{z})\right] \qquad \text{Introduce inference}$$

$$\geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\left[\log\left(\frac{p(\mathbf{z})}{q(\mathbf{z})} p(\mathbf{x}|\mathbf{z})\right)\right] \qquad \text{Use Jensens}$$

$$= \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{z}) - \log q(\mathbf{z}|\mathbf{x}) + \log p(\mathbf{x}|\mathbf{z})] \qquad \text{Simplify}$$

ELBO (without KL)

General Radon-Nikodym derivative

$$\mathbb{E}_p[f] = \mathbb{E}_q\left[\frac{dp}{dq}f\right]$$

Then

$$\begin{aligned}
\log p(\mathbf{x}) &= \log \mathbb{E}_{p(\mathbf{z})}[p(\mathbf{x}\,|\,\mathbf{z})] && \text{Marginal likelihood} \\
&= \log \mathbb{E}_{q(\mathbf{z}\,|\,\mathbf{x})}\left[\frac{p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})}p(\mathbf{x}\,|\,\mathbf{z})\right] && \text{Introduce inference} \\
&= \log \mathbb{E}_{q(\mathbf{z}\,|\,\mathbf{x})}\left[\frac{dp}{dq}p(\mathbf{x}\,|\,\mathbf{z})\right] && \text{Write with Radon-Nikodym} \\
&\geq \mathbb{E}_{q(\mathbf{z}\,|\,\mathbf{x})}\left[\log\left(\frac{p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})}p(\mathbf{x}\,|\,\mathbf{z})\right)\right] && \text{Use Jensens} \\
&= \mathbb{E}_{q(\mathbf{z}\,|\,\mathbf{x})}[\log p(\mathbf{z}) - \log q(\mathbf{z}\,|\,\mathbf{x}) + \log p(\mathbf{x}\,|\,\mathbf{z})] && \text{Simplify}
\end{aligned}$$

Girsanov: Introduce Inference

- Write a new process with variational "parameters" $v_{\boldsymbol{\lambda}}$
- Change the expectation to a new process (measure) with the Radon-Nikodym derivative

$$p(\mathbf{a}, 1-\boldsymbol{\epsilon}) = \mathbb{E}\left[p_0(\mathbf{z}_1) \exp\left(\int_{\boldsymbol{\epsilon}}^{1} -\nabla \cdot h(\mathbf{z}_s, 1-s) ds \right) | \, \mathbf{z}_\epsilon = \mathbf{a} \right]$$

$$d\mathbf{z}_t = -h(\mathbf{z}, 1-t)dt + \sigma(1-t)dB_t$$

to provide more ways to compute

Girsanov: Introduce Inference

Call the distribution $\hat{q}$ for

$$d\mathbf{z}_t = -h(\mathbf{z}, 1-t)dt + \sigma(1-t)d\hat{B}_t$$
$$\mathbf{z}_\epsilon = \mathbf{a}$$

Take a distribution q

$$d\mathbf{z}_t = -h(\mathbf{z}_t, 1-t)dt + \sigma(1-t)v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)dt + \sigma(1-t)dB_t$$
$$\mathbf{z}_\epsilon = \mathbf{a}$$

The Radon-Nikodym derivative is

$$\frac{d\hat{q}}{dq} = \exp\left(-\int_\epsilon^1 v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)dB_t - \int_\epsilon^1 \frac{1}{2}||v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)||_2^2 dt\right)$$
$$\hat{B}_t = B_t + \int_0^t v_{\boldsymbol{\lambda}}(\mathbf{z}_s, s)ds$$

Provides a more flexible way to compute expectations under $\hat{q}$

Girsanov: Introduce Inference

Two ways to sample $\mathbf{z}$

- $\hat{q} : d\mathbf{z}_t = -h(\mathbf{z}, 1-t)dt + \sigma(1-t)d\hat{B}_t$
- $q : d\mathbf{z}_t = -h(\mathbf{z}_t, 1-t)dt + \sigma(1-t)v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)dt + \sigma(1-t)dB_t$

Now use Radon-Nikodym in log probability

$$
\begin{aligned}
p(\mathbf{a}, 1-\epsilon) &= \mathbb{E}_{\hat{q}}\left[p_0(\mathbf{z}_1)\exp\left(\int_{\epsilon}^{1} -\nabla \cdot h(\mathbf{z}_s, 1-s)ds\right) \mid \mathbf{z}_{\epsilon} = \mathbf{a}\right] \\
&= \mathbb{E}_{q}\left[\frac{d\hat{q}}{dq}p_0(\mathbf{z}_1)\exp\left(\int_{\epsilon}^{1} -\nabla \cdot h(\mathbf{z}_s, 1-s)ds\right) \mid \mathbf{z}_{\epsilon} = \mathbf{a}\right]
\end{aligned}
$$

with

$$
\frac{d\hat{q}}{dq} = \exp\left(-\int_{\epsilon}^{1} v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)dB_t - \int_{\epsilon}^{1} \frac{1}{2}||v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)||_2^2 dt\right)
$$

Jensen's: Lower Bound

$$
\begin{aligned}
&\log p(\mathbf{a}, 1-\epsilon) \\
&\quad = \log \mathbb{E}_q \left[\frac{d\hat{q}}{dq} p_0(\mathbf{z}_1) \exp\left(\int_{\epsilon}^{1} -\nabla \cdot h(\mathbf{z}_s, 1-s) ds \right) \mid \mathbf{z}_\epsilon = \mathbf{a} \right] \\
&\quad \geq \mathbb{E}_q \left[\log \frac{d\hat{q}}{dq} + \log p_0(\mathbf{z}_1) + \int_{\epsilon}^{1} -\nabla \cdot h(\mathbf{z}_s, 1-s) ds \mid \mathbf{z}_\epsilon = \mathbf{a} \right]
\end{aligned}
$$

Lower Bound

Simplifying the first term

$$\begin{aligned}
&\mathbb{E}_q\left[\log \frac{d\hat{q}}{dq}\right] \\
&= \mathbb{E}_q\left[-\int_{\epsilon}^{1} v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t) dB_t - \int_{\epsilon}^{1} \frac{1}{2}||v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)||_2^2 dt\right] \\
&= \mathbb{E}_q\left[-\frac{1}{2}\int_{\epsilon}^{1} ||v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)||_2^2 dt\right]
\end{aligned}$$

Integrals of adapted functions with respect to Brownian motion on average are zero

Jensen's: Lower Bound

$$\log p(\mathbf{a}, 1-\epsilon)$$

$$\geq \mathbb{E}_q \left[\int_{\epsilon}^{1} \left(-\frac{1}{2} ||v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)||_2^2 - \nabla \cdot h(\mathbf{z}_t, 1-t) \right) dt + \log p_0(\mathbf{z}_{1-\epsilon}) \,|\, \mathbf{z}_{\epsilon} = \mathbf{a} \right]$$

$$d\mathbf{z}_t = -h(\mathbf{z}_t, 1-t)dt + \sigma(1-t)v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)dt + \sigma(1-t)dB_t$$

- Everything is computable
- Variational approximation has parameters $\boldsymbol{\lambda}$
- Model is in h can be parameterized by $\boldsymbol{\theta}$

Simplify

$$\log p(\mathbf{a}, 1-\epsilon)$$

$$\geq \mathbb{E}_q\left[\int_{\epsilon}^{1}\left(-\frac{1}{2}||v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)||_2^2 - \nabla \cdot h(\mathbf{z}_t, 1-t)\right)dt + \log p_0(\mathbf{z}_{1-\epsilon}) \,|\, \mathbf{z}_{\epsilon} = \mathbf{a}\right]$$

$$d\mathbf{z}_t = -h(\mathbf{z}_t, 1-t)dt + \sigma(1-t)v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t)dt + \sigma(1-t)dB_t$$

- Pick $h(\mathbf{z}_t, 1-t) = \sigma(1-t)\sigma(1-t)^{\top}s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t) - f(\mathbf{z}_t, t)$
- Pick $v_{\boldsymbol{\lambda}}(\mathbf{z}_t, t) = \sigma(1-t)^{\top}s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t)$

yields

$$\int_{\epsilon}^{1}\mathbb{E}_q\Big[-\frac{1}{2}||s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t)||^2_{\sigma(1-t)\sigma(1-t)^{\top}}$$

$$-\nabla \cdot \left[\sigma(1-t)\sigma(1-t)^{\top}s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t) - f(\mathbf{z}_t, t)\right] \,|\, \mathbf{z}_{\epsilon} = \mathbf{a}\Big]dt$$

$$+\mathbb{E}_q[\log p_0(\mathbf{z}_1) \,|\, \mathbf{z}_{\epsilon} = \mathbf{a}]$$

$$d\mathbf{z}_t = f(\mathbf{z}, t)dt + \sigma(1-t)dB_t$$

Lower bound for $\log p(\mathbf{z}_\epsilon) = \log p(\mathbf{a}, 1-\epsilon)$

$$\mathbb{E}_{\mathbf{a}\sim q(\mathbf{z}_\epsilon \mid \mathbf{x})} \log p(\mathbf{a}, 1-\epsilon) = \mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})} \log p(\mathbf{z}_\epsilon)$$

$$\geq \int_\epsilon^1 \mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})} \mathbb{E}_q \Big[-\frac{1}{2} ||s_\theta(\mathbf{z}_t, 1-t)||^2_{\sigma(1-t)\sigma(1-t)^\top}$$

$$- \nabla \cdot \big[\sigma(1-t)\sigma(1-t)^\top s_\theta(\mathbf{z}_t, 1-t) - f(\mathbf{z}_t, t)\big] \mid \mathbf{z}_\epsilon \Big] dt$$

$$+ \mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})} \mathbb{E}_q [\log p_0(\mathbf{z}_1) \mid \mathbf{z}_\epsilon]$$

$$d\mathbf{z}_t = f(\mathbf{z}, t)dt + \sigma(1-t)dB_t$$

Pick $q(\mathbf{z}_\epsilon \mid \mathbf{x})$ to match

$$d\mathbf{z}_t = f(\mathbf{z}, t)dt + \sigma(1-t)dB_t, \mathbf{z}_0 = \mathbf{x}$$

Then

$$\mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})} \mathbb{E}_{q(\mathbf{z}_t \mid \mathbf{z}_\epsilon)}[g] = \mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{z}_0 = \mathbf{x})} \mathbb{E}_{q(\mathbf{z}_t \mid \mathbf{z}_\epsilon)}[g]$$

$$= \mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{z}_0 = \mathbf{x})} \mathbb{E}_{q(\mathbf{z}_t \mid \mathbf{z}_\epsilon, \mathbf{z}_0)}[g] = \mathbb{E}_{q(\mathbf{z_t} \mid \mathbf{z}_0 = \mathbf{x})}[g]$$

Expectation by parts/Stein's Lemma

$$\mathbb{E}_q[\nabla \cdot f(\mathbf{z})] = -\mathbb{E}_q[\nabla_{\mathbf{z}} \log q(\mathbf{z})^\top f(\mathbf{z})]$$

Can be proved using integration by parts

Simplify

$$\int_{\epsilon}^{1} \mathbb{E}_q \Bigg[-\frac{1}{2} ||s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t)||^2_{\sigma(1-t)\sigma(1-t)^\top}$$

$$-\nabla \cdot \left[\sigma(1-t)\sigma(1-t)^\top s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t) - f(\mathbf{z}_t, t)\right] | \, \mathbf{z}_0 = \mathbf{x} \Bigg] dt$$

$$= \int_{\epsilon}^{1} \mathbb{E}_q \Bigg[-\frac{1}{2} ||s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t)||^2_{\sigma(1-t)\sigma(1-t)^\top}$$

$$+ \nabla \log q(\mathbf{z}_t | \, \mathbf{z}_0)^\top \left[\sigma(1-t)\sigma(1-t)^\top s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t) - f(\mathbf{z}_t, t)\right] | \, \mathbf{z}_0 = \mathbf{x} \Bigg] dt$$

$$= \int_{\epsilon}^{1} \mathbb{E}_q \Bigg[-\frac{1}{2} ||s_{\boldsymbol{\theta}}(\mathbf{z}_t, 1-t) - \nabla \log q(\mathbf{z}_t | \, \mathbf{z}_0)||^2_{\sigma(1-t)\sigma(1-t)^\top}$$

$$-\nabla \log q(\mathbf{z}_t | \, \mathbf{z}_0)^\top f(\mathbf{z}_t, t) + \frac{1}{2} ||\log \nabla q(\mathbf{z}_t | \, \mathbf{z}_0)||^2_{\sigma(1-t)\sigma(1-t)^\top} | \, \mathbf{z}_0 = \mathbf{x} \Bigg] dt$$

$$d\mathbf{z}_t = f(\mathbf{z}, t)dt + \sigma(1-t)dB_t$$

Putting it all together

Lower bound for $\log p(\mathbf{z}_\epsilon)$

$$\mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})}[\log p(\mathbf{z}_\epsilon)] \geq \mathbb{E}_q[\log p_0(\mathbf{z}_{1-\epsilon}) \mid \mathbf{z}_0 = \mathbf{a}]$$

$$\int_\epsilon^1 \mathbb{E}_q\left[-\frac{1}{2}||s_{\boldsymbol{\theta}} - \nabla lq_{t\mid 0}||^2_{\sigma\sigma^\top} - \nabla lq_{t\mid 0}^\top f + \frac{1}{2}||lq_{t\mid 0}||^2_{\sigma\sigma^\top} dt \mid \mathbf{z}_0 = \mathbf{x} \right]$$

$$d\mathbf{z}_t = f(\mathbf{z}, t)dt + \sigma(1-t)dB_t$$

A few comments:

- Full ELBO

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})}[\log p(\mathbf{x} \mid \mathbf{z}_\epsilon) + \log p(\mathbf{z}_\epsilon) - \log q(\mathbf{z}_\epsilon \mid \mathbf{x})]$$

- Parameters only appear in $\mathbb{E}_q\left[-\frac{1}{2}||s_{\boldsymbol{\theta}} - \nabla lq_{t\mid 0}||^2_{\sigma\sigma^\top}\right]$
- This is a type of **score-matching**
- *A good approximation picks f so distribution goes to p_0*

Architectures + Training

$$\mathbb{E}_{q(\mathbf{z}_\epsilon \mid \mathbf{x})}[\log p(\mathbf{z}_\epsilon)] \geq \mathbb{E}_q[\log p_0(\mathbf{z}_1) \mid \mathbf{z}_0 = \mathbf{x}]$$

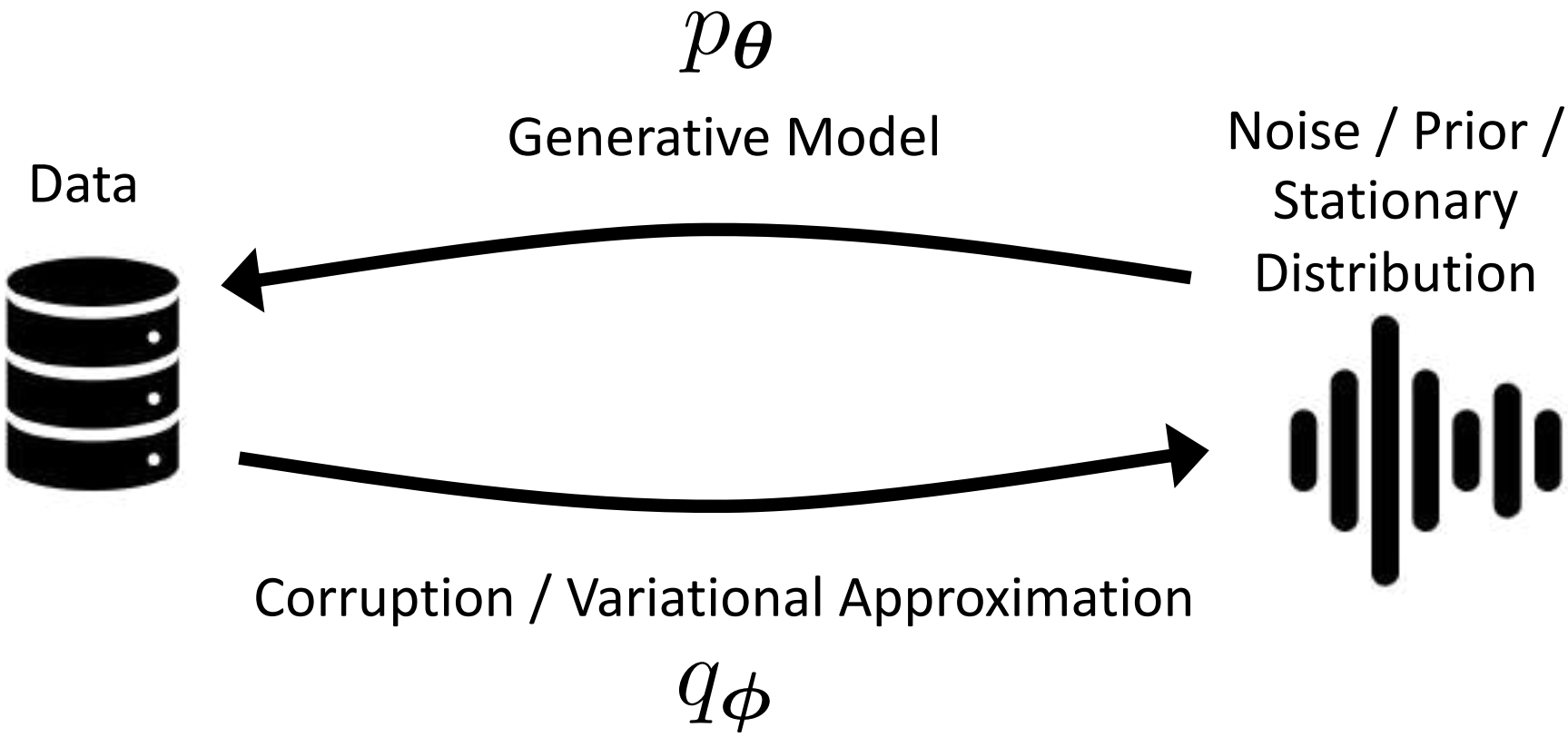
$$\int_\epsilon^1 \mathbb{E}_q\left[-\frac{1}{2}||s_{\boldsymbol{\theta}} - \nabla lq_{t\mid 0}||^2_{\sigma\sigma^\top} - \nabla lq_{t\mid 0}^\top f + \frac{1}{2}||lq_{t\mid 0}||^2_{\sigma\sigma^\top} \mid \mathbf{z}_0 = \mathbf{x}\right] dt$$

Score models $s_{\boldsymbol{\theta}}(\mathbf{z}_t, t)$ have the same input and output dimension

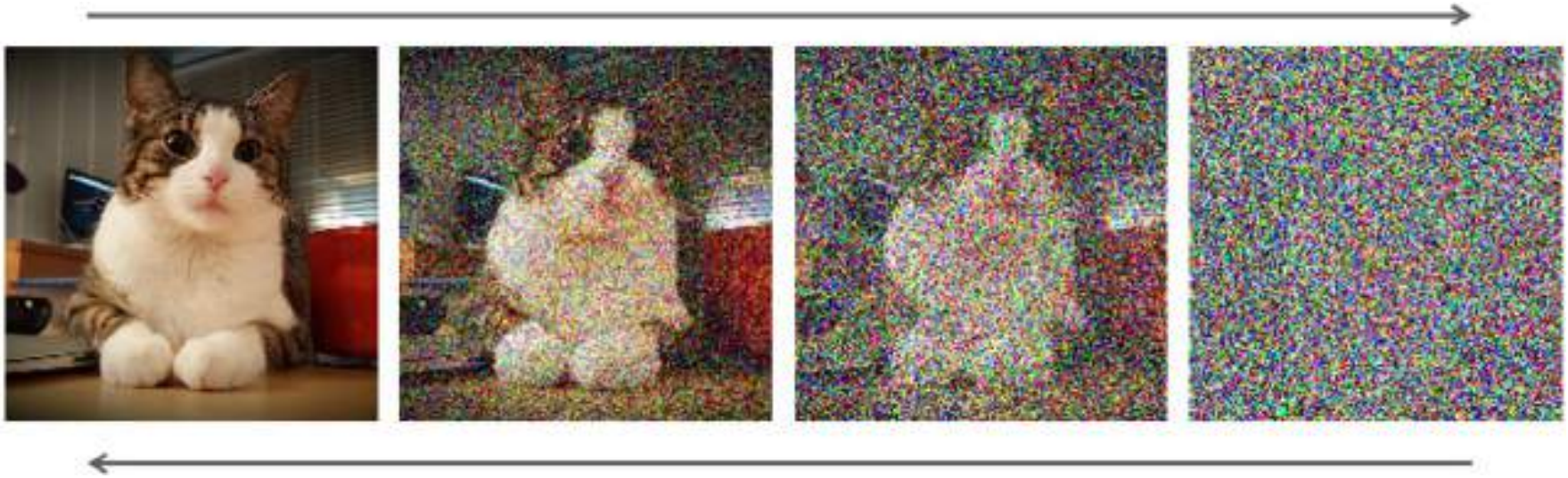
- Use a U-Net originally designed to segment images
- Use a Transformer stack (more on transformers later)
- Model size often requires training on multiple GPUs
- Data often reduced to lower dimension for tractability

Train with stochastic gradient optimization!

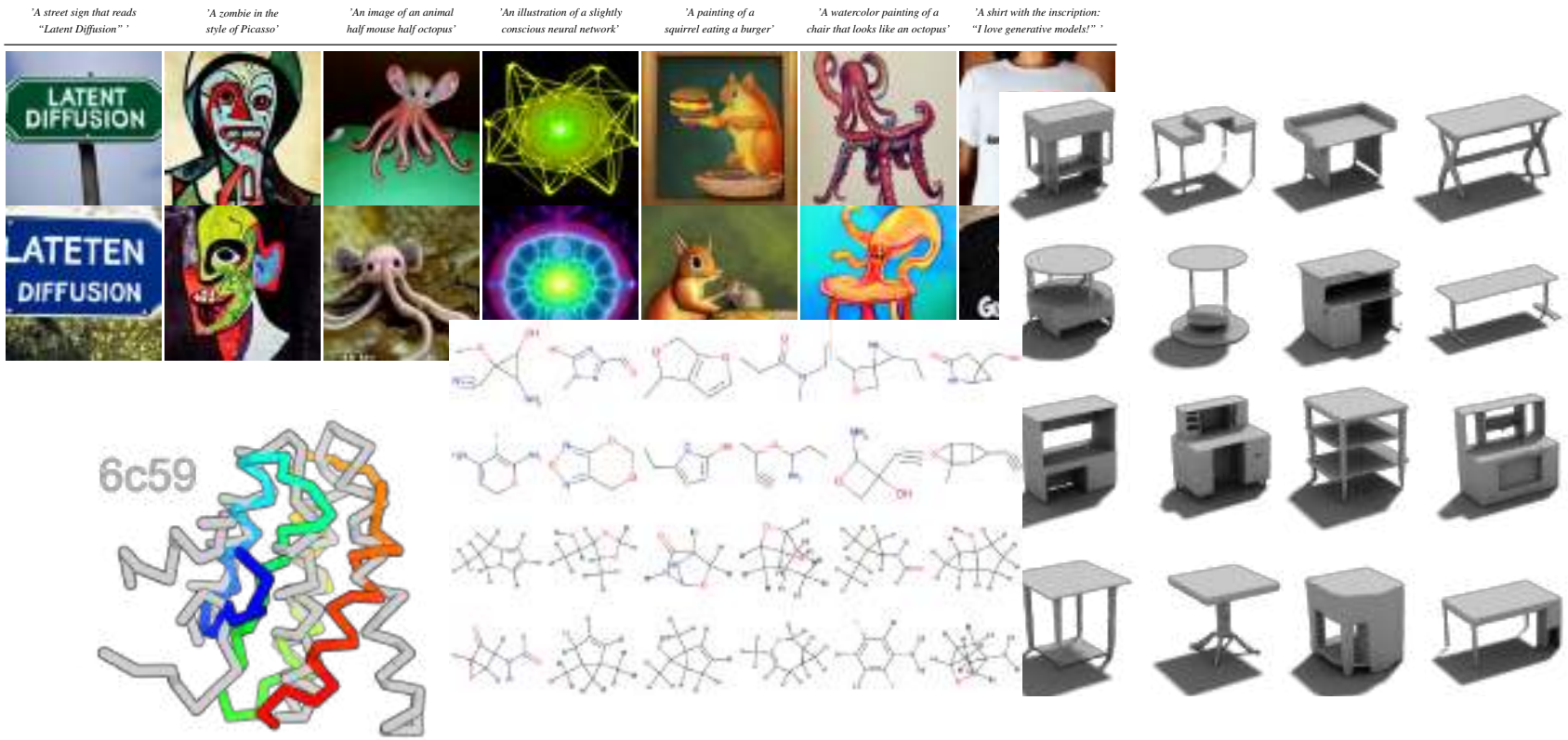
High Level View



High Level View



Generation Results



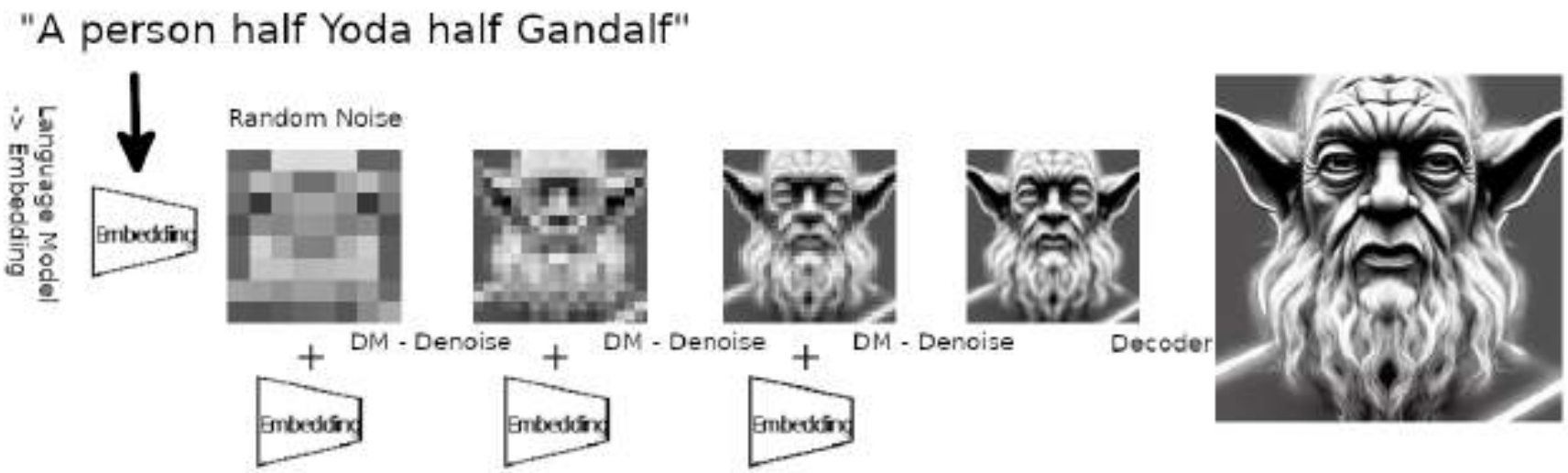
[Rombach+ 2022, Trippe+ 2023, Liu+ 2023, Vignac+ 2023]

Conditional Generation

Goal is to generate from $p(\mathbf{x}|\mathbf{y})$

- $\mathbf{y}$ could be text
- $\mathbf{x}$ could be an image

Parameterize score models s_{θ} to take in the conditioning $\mathbf{y}$



[paepper]



Sprouts in the shape of text 'Imagen' coming out of a fairytale book.



A photo of a Shiba Inu dog with a backpack riding a bike. It is wearing sunglasses and a beach hat.



A high contrast portrait of a very happy fuzzy panda dressed as a chef in a high end kitchen making dough. There is a painting of flowers on the wall behind him.



Teddy bears swimming at the Olympics 400m Butterfly event.



A cute corgi lives in a house made out of sushi.



A cute sloth holding a small treasure chest. A bright golden glow is coming from the chest.

[Synced]

Want to generate samples with some $p(\mathbf{x})$

Neural networks can transform unstructured noise to structured observations

Variational autoencoders do it with noise greater than the dimension of the data

Direct likelihood estimation does it with noise equal in dimension to the data

New fun generative models that use diffusions

Diffusions do it with stochastic processes

How could we use deep generative models to create a new molecule to treat knee pain?